

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO TÌM HIỂU CÔNG NGHỆ AI

Giảng viên hướng dẫn: TS Đỗ Thị Liên

Nhóm thực hiện:

Nguyễn Đức Bảo B22DCCN064

Nguyễn Mạnh Cường B22DCCN100

Lớp: D22CNPM02

Hà Nội, tháng 7 năm 2026

Mục lục

CHƯƠNG I: ĐẶT VẤN ĐỀ VÀ MỤC TIÊU.....	3
1.1. Đặt vấn đề	3
1.2. Mục tiêu nghiên cứu	3
1.3. Phạm vi nghiên cứu	4
CHƯƠNG II: TỔNG QUAN HỆ TƯ VẤN	4
2.1. Định nghĩa và mô hình hóa bài toán tổng quát	4
2.2. Phân loại các hướng tiếp cận	5
CHƯƠNG III: CÁC PHƯƠNG PHÁP TƯ VẤN.....	6
3.1. Phương pháp Lọc nội dung (Content-Based Filtering).....	6
3.2. Collaborative Filtering — Phân rã ma trận (Matrix Factorization)	10
3.3. Phương pháp Lai (Hybrid Systems — Late Fusion).....	14
3.4. Mạng nơ-ron đồ thị (LightGCN — Đồ thị lưỡng phân)	17
CHƯƠNG IV: CÀI ĐẶT HỆ THỐNG	23
4.1. Kiến trúc hệ thống tổng thể.....	23
4.2. Luồng thực thi phương pháp Lọc nội dung	24
4.3. Luồng thực thi Phân rã ma trận.....	25
4.4. Luồng thực thi Phương pháp Lai	27
4.5. Luồng thực thi Mạng nơ-ron đồ thị LightGCN	28
4.6. Quy trình Huấn luyện và Đánh giá tổng thể	32
CHƯƠNG V: THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	34
5.1. Thiết lập thực nghiệm và Đặc tả các Bộ dữ liệu.....	34
5.1.1 Tổng quan 3 bộ dữ liệu chuẩn quốc tế:.....	34
5.1.2 Bảng đặc tả thông số thống kê đối chiếu:	34
5.1.3 Quy trình tiền xử lý dữ liệu:.....	35
5.1.4 Thiết lập Tham số hệ thống:	35
5.2. Hệ thống các Độ đo Đánh giá Tiêu chuẩn	36
5.3. Kết quả thực nghiệm trên MovieLens	36
5.4. Kết quả thực nghiệm trên Last.fm	38
5.5. Kết quả thực nghiệm trên Amazon Musical Instruments	39
5.6. Phân tích So sánh Tổng hợp và Đánh giá Tính Tổng quát	40

CHƯƠNG I: ĐẶT VẤN ĐỀ VÀ MỤC TIÊU

1.1. Đặt vấn đề

Trong kỷ nguyên số hiện nay, sự phát triển mạnh mẽ của Internet và các nền tảng công nghệ đã tạo ra một khối lượng dữ liệu khổng lồ. Điều này dẫn đến một vấn đề phổ biến được gọi là hiện tượng "quá tải thông tin". Người dùng khi truy cập vào các nền tảng thương mại điện tử, dịch vụ giải trí hay mạng xã hội thường xuyên phải đối mặt với hàng triệu sản phẩm, nội dung, hoặc dịch vụ (được gọi chung là các Item). Việc phải tự tìm kiếm và chọn lọc ra những Item thực sự phù hợp với nhu cầu và sở thích cá nhân giữa một "biển" thông tin trở nên vô cùng khó khăn và mất thời gian.

Từ góc độ của các nhà cung cấp dịch vụ, nếu không thể giúp người dùng nhanh chóng tìm thấy những Item họ mong muốn, nền tảng sẽ dễ dàng đánh mất khách hàng, làm giảm trải nghiệm người dùng và ảnh hưởng trực tiếp đến doanh thu.

Để giải quyết bài toán này, Hệ tư vấn (Recommender Systems) ra đời như một giải pháp công nghệ tất yếu. Hệ tư vấn đóng vai trò như một bộ lọc thông minh, tự động phân tích hành vi trong quá khứ, sở thích và ngữ cảnh của người dùng để trích xuất ra một danh sách các Item (Top-K Items) có khả năng được người dùng quan tâm nhất. Một hệ tư vấn tốt không chỉ giúp người dùng tiết kiệm thời gian tìm kiếm mà còn giúp nền tảng tăng cường độ gắn kết (user engagement) và tối ưu hóa lợi nhuận.

1.2. Mục tiêu nghiên cứu

Xuất phát từ nhu cầu thực tiễn của bài toán gợi ý Item cho người dùng, đồ án này được thực hiện với các mục tiêu cụ thể như sau:

- Nghiên cứu cơ sở lý thuyết: Tìm hiểu sâu về kiến trúc tổng quát của hệ tư vấn và các phương pháp tiếp cận phổ biến hiện nay, bao gồm: Lọc theo nội dung (Content-Based Filtering), Lọc cộng tác (Collaborative Filtering), Phương pháp Lai (Hybrid).
- Nghiên cứu phương pháp nâng cao: Đi sâu tìm hiểu và mô hình hóa phương pháp mạng nơ-ron đồ thị (Graph Neural Networks - GNN), cụ thể là mô hình LightGCN để khai thác thông tin lan truyền trên đồ thị lưỡng phân.
- Cài đặt và thực nghiệm: Triển khai cài đặt mã nguồn cho các phương pháp kể trên. Tiến hành huấn luyện và kiểm thử các mô hình trên một bộ dữ liệu tiêu chuẩn đã được công bố rộng rãi trong giới học thuật.
- Đánh giá và so sánh: Sử dụng các độ đo quy chuẩn (Precision, Recall, NDCG...) để so sánh hiệu năng giữa các mô hình, từ đó đưa ra phân tích, giải thích về tính hiệu quả của từng phương pháp đối với bài toán gợi ý.

1.3. Phạm vi nghiên cứu

Để đảm bảo tính tập trung và độ chính xác của thực nghiệm, báo cáo giới hạn phạm vi nghiên cứu ở các khía cạnh sau:

- Về thuật toán: Tập trung triển khai các thuật toán đại diện cho từng phương pháp (Cosine Similarity cho Content-Based, Matrix Factorization cho Lọc cộng tác, Late Fusion cho Hybrid và LightGCN cho mạng đồ thị).
- Về dữ liệu: Đồ án không thu thập dữ liệu thô từ môi trường thực tế (như các website đặc thù) mà sử dụng bộ dữ liệu công khai chuẩn mực (ví dụ: MovieLens) nhằm đảm bảo tính khách quan, có khả năng tái tạo (reproducibility) và tránh các nhiễu dữ liệu không mong muốn.
- Về đánh giá: Chỉ tập trung đánh giá hiệu năng (accuracy, ranking quality) của hệ thống thông qua các công thức toán học và độ đo định lượng, không đánh giá qua khảo sát người dùng thực tế (A/B Testing) do giới hạn về mặt thời gian và tài nguyên.

CHƯƠNG II: TỔNG QUAN HỆ TƯ VẤN

2.1. Định nghĩa và mô hình hóa bài toán tổng quát

Hệ tư vấn Recommender System là một dạng hệ thống lọc thông nhằm dự đoán "đánh giá" (rating) hoặc "mức độ ưu tiên" (preference) mà một người dùng sẽ dành cho một sản phẩm. Mục đích cốt lõi của hệ thống là tự động thu hẹp không gian tìm kiếm, mang lại những gợi ý mang tính cá nhân hóa cao nhất.

Dưới góc độ toán học, bài toán gợi ý có thể được mô hình hóa tổng quát như sau:

- **Tập hợp Người dùng (Users):** Ký hiệu là $U = \{u_1, u_2, \dots, u_M\}$, trong đó M là tổng số người dùng.
- **Tập hợp Sản phẩm (Items):** Ký hiệu là $I = \{i_1, i_2, \dots, i_N\}$, trong đó N là tổng số sản phẩm.
- **Lịch sử tương tác (Interactions):** Được biểu diễn bằng một ma trận tương tác R kích thước $M \times N$. Một phần tử $r_{u,i}$ trong ma trận thể hiện mức độ tương tác của người dùng u lên sản phẩm i . Tương tác này có thể là dạng tường minh (Explicit feedback - ví dụ: chấm điểm 1-5 sao) hoặc dạng ẩn (Implicit feedback - ví dụ: click xem, thả tim, với giá trị 1 là đã tương tác và 0 là chưa tương tác).

Input:

- Ma trận tương tác R giữa tập User U và tập Item I .

- (Tùy chọn) Dữ liệu ngữ cảnh bổ sung: Các đặc trưng văn bản, danh mục của Item, hoặc thông tin cá nhân của User.

Output:

- Với một người dùng u cụ thể, hệ thống phải dự đoán mức độ quan tâm $\hat{y}_{u,i}$ đối với các Item i mà người dùng đó chưa từng tương tác.
- Đầu ra cuối cùng là một danh sách Top-K Items có điểm dự đoán $\hat{y}_{u,i}$ cao nhất, được sắp xếp giảm dần để gợi ý trực tiếp cho người dùng u .

2.2. Phân loại các hướng tiếp cận

Để giải quyết bài toán tính điểm dự đoán $\hat{y}_{u,i}$, các hệ tư vấn hiện đại được chia thành 4 nhánh tiếp cận chính:

1. Lọc theo nội dung:

- Nguyên lý: "Nếu trong quá khứ bạn thích Item A, hệ thống sẽ gợi ý cho bạn Item B có nội dung/đặc trưng tương tự với A".
- Đặc điểm: Phương pháp này chỉ quan tâm đến lịch sử của cá nhân người dùng đó và các mô tả (features, tags) của Item, hoàn toàn bỏ qua dữ liệu hành vi của những người dùng khác trong hệ thống.

2. Lọc cộng tác:

- Nguyên lý: "Nếu những người có sở thích giống bạn thích Item A, thì nhiều khả năng bạn cũng sẽ thích Item A".
- Đặc điểm: Không cần biết Item đó có nội dung gì, phương pháp này thuần túy dựa vào ma trận tương tác User-Item để khai phá các hành vi chung của cộng đồng. Đại diện tiêu biểu nhất và mạnh mẽ nhất của nhánh này là các mô hình **Phân rã ma trận (Matrix Factorization - MF)**, nơi ma trận thưa ban đầu được ép xuống thành các vector đặc trưng ẩn (Latent Embeddings).

3. Phương pháp Lai:

- Nguyên lý: Kết hợp thế mạnh của cả Lọc nội dung và Lọc cộng tác để bù trừ những nhược điểm của nhau.
- Đặc điểm: Phương pháp lai giúp giải quyết vấn đề "Khởi động lạnh" (khi hệ thống chưa có dữ liệu tương tác) của Lọc cộng tác bằng cách dùng tín hiệu văn bản của Lọc nội dung để hỗ trợ chấm điểm tổng hợp.

4. Mạng nơ-ron đồ thị:

- Nguyên lý: Xem dữ liệu gợi ý dưới dạng một **Đồ thị lưỡng phân User–Item**. Người dùng và Sản phẩm là các đỉnh (Nodes), tương tác giữa chúng là các cạnh (Edges).
- Đặc điểm: GNN cho phép "lan truyền thông tin" qua nhiều bước trên đồ thị. Điều này giúp hệ thống học được các mối quan hệ bậc cao, ví dụ: "Bạn bè của những người giống bạn đang tương tác với Item nào", giúp vector biểu diễn của User và Item trở nên cực kỳ chính xác.

CHƯƠNG III: CÁC PHƯƠNG PHÁP TƯ VẤN

Chương này trình bày chi tiết cơ sở lý thuyết và cơ chế hoạt động của 4 phương pháp tư vấn đại diện: Lọc nội dung, Lọc cộng tác Phân rã ma trận (Matrix Factorization), Phương pháp Lai kết hợp muộn và Mạng nơ-ron đồ thị (LightGCN trên Đồ thị lưỡng phân). Với mỗi phương pháp, báo cáo mô hình hóa mạch lạc từ ý tưởng bản chất, định nghĩa đầu vào/đầu ra, cấu trúc luồng xử lý trên sơ đồ, hệ thống công thức toán học cho đến ví dụ tính toán bằng số thực tế.

3.1. Phương pháp Lọc nội dung (Content-Based Filtering)

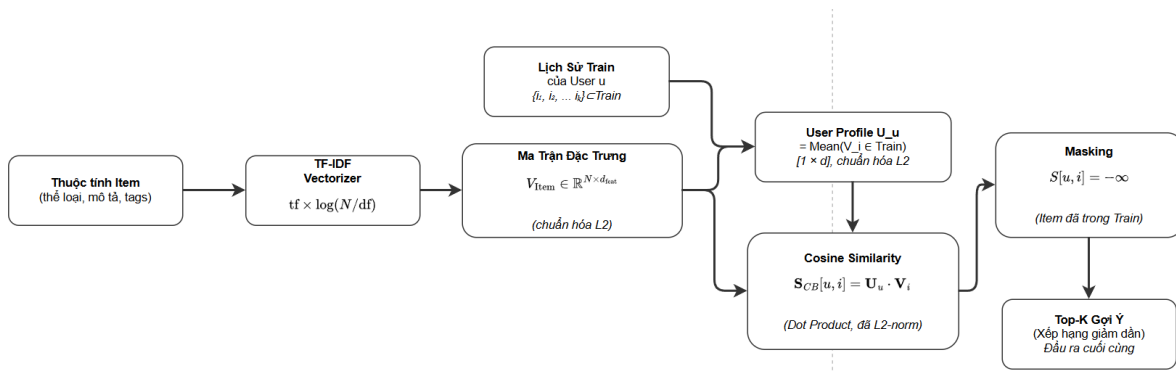
a. Ý tưởng bản chất

Phương pháp Lọc nội dung dựa trên nguyên lý so sánh độ tương đồng đặc trưng: hệ thống gợi ý cho người dùng những Item có đặc trưng nội dung (từ khóa, thể loại, mô tả) tương tự với các *Item* mà *User* đó đã từng tương tác và đánh giá tích cực trong quá khứ. Phương pháp này tập trung khai thác lịch sử cá nhân của *User* và nội dung của *Item*, hoàn toàn độc lập với hành vi tương tác của cộng đồng *User* khác.

b. Dữ liệu Đầu vào / Đầu ra

- Đầu vào (Input):
 - Tập thuộc tính văn bản/mô tả của toàn bộ các *Item* trong hệ thống.
 - Lịch sử tương tác của *User* u (danh sách tập hợp các *Item* đã tương tác tích cực I_{u+}).
- Đầu ra (Output):
 - Điểm số tương đồng $\text{Score}(u, i) \in [0,1]$ cho các *Item* i chưa tương tác.
 - Danh sách *Top – K Item* có điểm tương đồng cao nhất xếp theo thứ tự giảm dần.

c. Bản chất quá trình xử lý và Sơ đồ luồng



Hình 1: Flowchart luồng xử lý Content-Based

Giải thích chi tiết các bước trên sơ đồ

- **Trích xuất đặc trưng Item (V_i):**

Đầu tiên, hệ thống thu thập các thông tin nội dung của từng Item như tên, mô tả, danh mục, từ khóa hoặc các thể thuộc tính. Dữ liệu văn bản thô được tiền xử lý bằng cách chuẩn hóa chữ viết, loại bỏ ký tự đặc biệt, từ không mang nhiều ý nghĩa và tách nội dung thành các từ khóa. Sau đó, kỹ thuật TF-IDF được sử dụng để chuyển văn bản thành vector số học V_i . Trong vector này, mỗi chiều tương ứng với một từ khóa trong bộ từ điển chung, còn giá trị tại mỗi chiều thể hiện mức độ quan trọng của từ khóa đó đối với Item. Một từ khóa xuất hiện nhiều trong một Item nhưng ít xuất hiện ở các Item khác sẽ có trọng số TF-IDF cao. Nhờ đó, vector V_i có thể biểu diễn được những đặc điểm nội dung nổi bật của Item và được sử dụng trong các phép tính so sánh tiếp theo.

- **Xây dựng hồ sơ người dùng (U_u):**

Sau khi các Item đã được biểu diễn thành vector, hệ thống tiến hành xây dựng hồ sơ sở thích cho từng người dùng. Với người dùng u , tập I_{u+} bao gồm những Item mà người dùng đã có tương tác tích cực, chẳng hạn như đã xem, yêu thích, đánh giá cao, lựa chọn hoặc lưu lại. Hệ thống thu thập các vector V_i của toàn bộ Item thuộc tập I_{u+} , sau đó tính trung bình cộng để tạo ra vector hồ sơ người dùng U_u . Vector này tổng hợp những đặc trưng thường xuất hiện trong các Item mà người dùng quan tâm. Nếu một nhóm từ khóa có trọng số cao trong nhiều Item đã tương tác, các thành phần tương ứng trong U_u cũng sẽ có giá trị lớn. Vì vậy, U_u được xem là biểu diễn tổng quát về sở thích nội dung hoặc “gu” của người dùng trong không gian đặc trưng TF-IDF.

- **Chấm điểm tương đồng giữa người dùng và Item:**

Sau khi có vector hồ sơ U_u , hệ thống tính mức độ phù hợp giữa người dùng và từng Item ứng viên bằng độ tương đồng Cosine. Các Item ứng viên thường là những Item mà người dùng chưa từng tương tác. Cosine Similarity so sánh hướng của vector U_u với vector nội dung V_i , thay vì chỉ so sánh độ lớn tuyệt đối của hai vector. Nếu hai vector có hướng gần giống nhau, góc giữa chúng nhỏ và điểm Cosine sẽ tiến gần đến 1. Điều này cho thấy Item có nhiều đặc trưng nội dung trùng khớp với sở thích của người dùng. Ngược lại, nếu

điểm Cosine gần 0, Item có ít đặc trưng chung với hồ sơ người dùng. Việc sử dụng Cosine Similarity cũng giúp hạn chế ảnh hưởng của độ dài văn bản, vì một Item có phần mô tả dài hơn không nhất thiết được đánh giá là phù hợp hơn.

- **Lọc, xếp hạng và lựa chọn Top-K:**

Sau khi tính điểm tương đồng cho các Item ứng viên, hệ thống áp dụng bộ lọc để loại bỏ những Item mà người dùng đã tương tác trước đó. Bước này giúp tránh tình trạng gợi ý lặp lại các Item đã được xem, lựa chọn hoặc đánh giá. Các Item còn lại được sắp xếp theo điểm tương đồng từ cao xuống thấp. Item có điểm lớn hơn sẽ được đặt ở vị trí cao hơn vì được dự đoán phù hợp hơn với sở thích của người dùng.

Cuối cùng, hệ thống lựa chọn k Item đứng đầu danh sách để tạo kết quả gợi ý Top-K. Danh sách này được xem là tập hợp các Item có nội dung phù hợp nhất với hồ sơ hiện tại của người dùng.

d. Công thức toán học

- Trọng số TF-IDF: Trích xuất đặc trưng văn bản của *Item d* qua từ khóa *t*:

$$W(t, d) = TF(t, d) \times IDF(t) = TF(t, d) \times \log\left(\frac{N}{DF(t)}\right)$$

Trong đó: *t* là từ khóa đang xét, *d* là Item chứa nội dung văn bản, $TF(t, d)$ biểu thị tần suất xuất hiện của từ khóa *t* trong Item *d*, *N* là tổng số Item trong tập dữ liệu và $DF(t)$ là số Item có chứa từ khóa *t*. Thành phần $IDF(t) = \log\left(\frac{N}{DF(t)}\right)$ dùng để giảm trọng số của những từ xuất hiện quá phổ biến và tăng trọng số cho các từ có khả năng phân biệt nội dung tốt hơn.

Công thức TF-IDF có vai trò chuyển nội dung văn bản của từng Item thành vector đặc trưng V_i . Mỗi phần tử trong vector là trọng số của một từ khóa, qua đó thể hiện những nội dung hoặc thuộc tính nổi bật của Item. Từ khóa xuất hiện nhiều trong Item nhưng ít xuất hiện ở các Item khác sẽ nhận trọng số cao hơn.

- Vector Hồ sơ người dùng (*User Profile*):

$$U_u = \frac{1}{|I_{u+}|} \sum_{i \in I_{u+}} V_i$$

Trong đó: U_u là vector hồ sơ sở thích của người dùng *u*, I_{u+} là tập hợp các Item mà người dùng đã có tương tác tích cực, $|I_{u+}|$ là số lượng Item trong tập đó và V_i là vector đặc

trung TF-IDF của Item i . Phép tổng lấy tất cả vector của các Item đã tương tác, sau đó chia cho số lượng Item để tính trung bình.

Công thức này có vai trò tổng hợp lịch sử tương tác thành một vector đại diện chung cho sở thích của người dùng. Những từ khóa hoặc đặc trưng xuất hiện thường xuyên trong các Item người dùng quan tâm sẽ có giá trị lớn trong U_u , từ đó giúp hệ thống xác định loại nội dung phù hợp với người dùng.

- Độ đo tương đồng Cosine Similarity:

$$\text{Score}(u, i) = \cos(U_u, V_i) = \frac{U_u \cdot V_i}{\|U_u\|_2 \cdot \|V_i\|_2} = \frac{\sum_k U_{u,k} \cdot V_{i,k}}{\sqrt{\sum_k U_{u,k}^2} \cdot \sqrt{\sum_k V_{i,k}^2}}$$

Trong đó: $\text{Score}(u, i)$ là điểm phù hợp giữa người dùng u và Item i , U_u là vector hồ sơ người dùng, V_i là vector đặc trưng của Item, $U_u \cdot V_i$ là tích vô hướng giữa hai vector, còn $\|U_u\|_2$ và $\|V_i\|_2$ là độ dài chuẩn L_2 của hai vector. Chỉ số k chạy qua tất cả các đặc trưng hoặc từ khóa trong không gian vector.

Công thức Cosine Similarity có vai trò đo mức độ giống nhau về hướng giữa hồ sơ người dùng và nội dung Item. Điểm càng gần 1 thì Item càng phù hợp với sở thích người dùng; điểm gần 0 cho thấy hai vector có ít đặc trưng chung. Sau khi tính điểm cho các Item chưa tương tác, hệ thống sắp xếp điểm giảm dần và chọn các Item có điểm cao nhất để tạo danh sách gợi ý Top- K .

Lưu ý: Trong trường hợp các vector U_u và V_i đã được chuẩn hóa L_2 từ trước để có độ dài bằng 1, tức $\|U_u\|_2 = 1$ và $\|V_i\|_2 = 1$, mẫu số $\|U_u\|_2 \cdot \|V_i\|_2 = 1$. Khi đó, công thức Cosine Similarity đồng nhất hoàn toàn với phép tính Tích vô hướng: $\text{Cosine}(U_u, V_i) = U_u \cdot V_i^T$.

e. Ví dụ bằng số cụ thể

- **Giả sử:** Không gian thuộc tính có 2 chiều $[Feature_1, Feature_2]$.
 - Item i_1 có vector TF-IDF: $V_1 = [0.8, 0.2]$
 - Item i_2 có vector TF-IDF: $V_2 = [0.1, 0.9]$
 - Item i_3 có vector TF-IDF: $V_3 = [0.6, 0.4]$
- Giả sử User u_1 mới chỉ tương tác với Item $i_1 \Rightarrow U_{u_1} = V_1 = [0.8, 0.2]$.
- Tính điểm dự đoán cho các Item chưa xem (i_2 và i_3):

- Với *Item* i_2 :
 - Tích vô hướng: $U_{u1} \cdot V_2 = (0.8 \times 0.1) + (0.2 \times 0.9) = 0.08 + 0.18 = 0.26$
 - Độ dài vector: $\|U_{u1}\| = \sqrt{0.8^2 + 0.2^2} \approx 0.8246$; $\|V_2\| = \sqrt{0.1^2 + 0.9^2} \approx 0.9055$
 - Điểm Cosine: $\text{Score}(u_1, i_2) = \frac{0.26}{0.8246 \times 0.9055} \approx 0.3484$
- Với *Item* i_3 :
 - Tích vô hướng: $U_{u1} \cdot V_3 = (0.8 \times 0.6) + (0.2 \times 0.4) = 0.48 + 0.08 = 0.56$
 - Độ dài vector: $\|V_3\| = \sqrt{0.6^2 + 0.4^2} \approx 0.7211$
 - Điểm Cosine: $\text{Score}(u_1, i_3) = \frac{0.56}{0.8246 \times 0.7211} \approx 0.9418$
- **Kết luận:** Do $\text{Score}(u_1, i_3) = 0.9418 > \text{Score}(u_1, i_2) = 0.3484$, hệ thống ưu tiên gợi ý *Item* i_3 cho *User* u_1 .

3.2. Collaborative Filtering — Phân rã ma trận (Matrix Factorization)

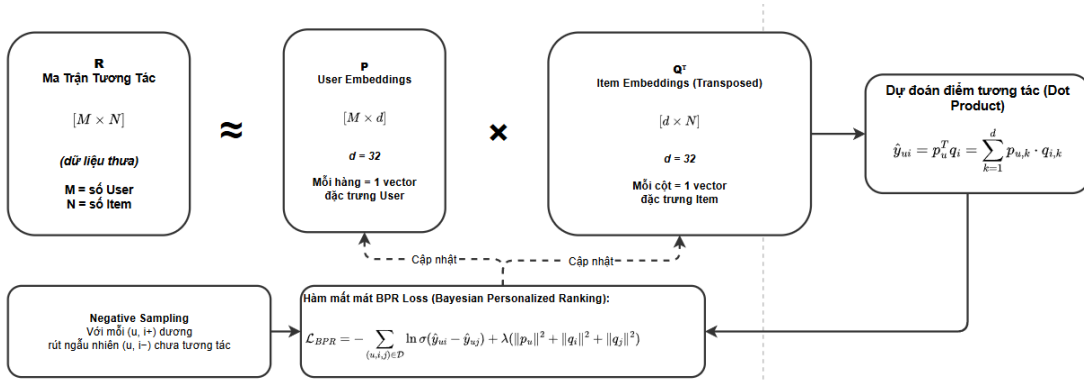
a. Ý tưởng bản chất

Phân rã ma trận (MF) học sở thích tiềm ẩn từ hành vi ứng xử chung của cộng đồng. Thay vì dùng thuộc tính văn bản, MF chiếu cả *User* và *Item* vào cùng một không gian đặc trưng ẩn mật độ thấp (kích thước d). Mức độ yêu thích của *User* đối với *Item* được mô hình hóa bằng tích vô hướng giữa 2 vector nhúng (Embedding).

b. Dữ liệu Đầu vào / Đầu ra

- Đầu vào (Input): Ma trận tương tác thưa $R \in \mathbb{R}^{M \times N}$ (M *User*, N *Item*), với $r_{ui} = 1$ nếu *User* u từng tương tác với *Item* i , và $r_{ui} = 0$ nếu chưa tương tác.
- Đầu ra (Output):
 - Ma trận nhúng *User* $P \in \mathbb{R}^{M \times d}$ chứa các vector p_u .
 - Ma trận nhúng *Item* $Q \in \mathbb{R}^{N \times d}$ chứa các vector q_i .
 - Điểm dự đoán tích vô hướng $\hat{y}_{ui} = p_u^T q_i$.

c. Bản chất quá trình xử lý và Sơ đồ luồng



Hình 2 : Sơ Đồ Phân Rã Ma Trận $R \approx P \times Q^T$

Giải thích chi tiết các bước trên sơ đồ

- Khởi tạo và chiếu dữ liệu vào không gian ẩn:**

Ma trận tương tác ban đầu R biểu diễn mối quan hệ giữa người dùng và Item, trong đó mỗi hàng tương ứng với một người dùng, mỗi cột tương ứng với một Item và các phần tử thể hiện trạng thái tương tác. Trong thực tế, ma trận này thường rất thưa vì mỗi người dùng chỉ tương tác với một số lượng nhỏ Item so với tổng số Item trong hệ thống. Để xử lý vấn đề này, mô hình Matrix Factorization phân rã thông tin tương tác thành hai ma trận có số chiều thấp hơn là P và Q . Mỗi người dùng u được gán một vector đặc trưng ẩn $p_u \in \mathbb{R}^d$, còn mỗi Item i được gán một vector đặc trưng ẩn $q_i \in \mathbb{R}^d$. Trong đó, d là số chiều của không gian ẩn và thường nhỏ hơn nhiều so với số lượng người dùng hoặc Item. Các vector này ban đầu được khởi tạo bằng những giá trị ngẫu nhiên nhỏ, sau đó được điều chỉnh trong quá trình huấn luyện. Các thành phần trong vector ẩn không nhất thiết biểu diễn trực tiếp những thuộc tính cụ thể như thể loại hay giá cả, mà được mô hình tự động học từ các mẫu tương tác để phản ánh sở thích tiềm ẩn của người dùng và đặc điểm tiềm ẩn của Item.

- Tái cấu trúc ma trận và dự đoán điểm bằng Dot Product:**

Sau khi có hai ma trận đặc trưng ẩn P và Q , mô hình thực hiện phép nhân $P \times Q^T$ để tái cấu trúc gần đúng ma trận tương tác ban đầu. Với một cặp người dùng u và Item i , điểm dự đoán được tính bằng tích vô hướng giữa hai vector p_u và q_i :

$$\hat{y}_{ui} = p_u^T q_i = \sum_{k=1}^d p_{u,k} q_{i,k}$$

Trong đó, $p_{u,k}$ là mức độ quan tâm của người dùng u đối với đặc trưng ẩn thứ k , còn $q_{i,k}$ thể hiện mức độ Item i mang đặc trưng đó. Nếu hai vector có nhiều thành phần tương ứng cùng mang giá trị lớn, tích vô hướng sẽ cao, cho thấy Item phù hợp với sở thích của người dùng. Ngược lại, điểm thấp thể hiện mức độ phù hợp thấp hơn. Nhờ phép tính này,

mô hình có thể dự đoán mức độ quan tâm của người dùng đối với cả những Item chưa từng tương tác, tức là các ô còn thiếu trong ma trận R . Sau khi tính điểm cho toàn bộ Item ứng viên, hệ thống có thể sắp xếp chúng theo điểm dự đoán giảm dần để xác định những Item có khả năng được người dùng quan tâm nhất.

- **Huấn luyện mô hình bằng BPR Loss:**

Để các vector p_u và q_i học được thứ tự ưu tiên phù hợp, mô hình sử dụng hàm mất mát Bayesian Personalized Ranking, gọi tắt là BPR Loss. Trong mỗi bước huấn luyện, hệ thống chọn một bộ ba (u, i, j) , trong đó u là người dùng, i là Item mà người dùng đã có tương tác tích cực và j là Item âm được lấy từ những Item người dùng chưa tương tác. Mô hình tính điểm $\hat{y}_{ui}$ cho Item tích cực và điểm $\hat{y}_{uj}$ cho Item âm, sau đó xét độ chênh lệch:

$$\hat{y}_{ui} - \hat{y}_{uj}$$

Mục tiêu của quá trình huấn luyện là làm cho hiệu điểm này càng lớn càng tốt, nghĩa là Item tích cực phải được xếp hạng cao hơn Item âm. BPR Loss sử dụng hàm Sigmoid để chuyển hiệu điểm thành mức độ tin cậy rằng người dùng ưu tiên Item i hơn Item j . Nếu $\hat{y}_{ui} > \hat{y}_{uj}$, giá trị mất mát giảm; ngược lại, nếu Item âm có điểm cao hơn, giá trị mất mát tăng và mô hình phải điều chỉnh lại các vector. Trong quá trình tối ưu, vector p_u được dịch chuyển gần hơn với q_i , đồng thời tách xa khỏi q_j . Thành phần chuẩn hóa L_2 cũng thường được bổ sung để giới hạn độ lớn của các vector và giảm nguy cơ quá khớp với dữ liệu huấn luyện.

d. Công thức toán học

- Dự đoán điểm tương tác (Dot Product):

$$\hat{y}_{ui} = p_u^T q_i = \sum_{k=1}^d p_{u,k} \cdot q_{i,k}$$

Trong đó: $\hat{y}_{ui}$ là điểm tương tác dự đoán giữa người dùng u và Item i ; $p_u \in \mathbb{R}^d$ là vector đặc trưng ẩn của người dùng; $q_i \in \mathbb{R}^d$ là vector đặc trưng ẩn của Item; d là số chiều của không gian ẩn. Hai đại lượng $p_{u,k}$ và $q_{i,k}$ lần lượt là thành phần thứ k trong vector người dùng và vector Item.

Công thức thực hiện phép nhân từng cặp thành phần tương ứng của p_u và q_i , sau đó cộng các kết quả để tạo thành điểm $\hat{y}_{ui}$. Điểm càng lớn cho thấy Item i càng phù hợp với sở thích của người dùng u . Công thức này có vai trò dự đoán mức độ quan tâm của người dùng đối với các Item chưa tương tác và làm cơ sở để sắp xếp danh sách gợi ý.

- Hàm mất mát BPR Loss (Bayesian Personalized Ranking):

$$\mathcal{L}_{BPR} = - \sum_{(u,i,j) \in \mathcal{D}} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda(\|p_u\|^2 + \|q_i\|^2 + \|q_j\|^2)$$

Trong đó: $\mathcal{D}$ là tập các bộ ba huấn luyện (u, i, j) ; u là người dùng; i là Item có tương tác tích cực; j là Item âm được lấy ngẫu nhiên từ những Item người dùng chưa tương tác. $\hat{y}_{ui}$ và $\hat{y}_{uj}$ lần lượt là điểm dự đoán của Item tích cực và Item âm. Hàm Sigmoid được xác định bởi:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Thành phần $\hat{y}_{ui} - \hat{y}_{uj}$ thể hiện mức chênh lệch thứ hạng giữa Item tích cực và Item âm. Khi hiệu điểm càng lớn, mô hình càng chắc chắn rằng người dùng ưu tiên Item i hơn Item j , vì vậy giá trị mất mát sẽ giảm. Ngược lại, nếu Item âm có điểm cao hơn, hàm mất mát tăng và buộc mô hình điều chỉnh lại các vector.

Thành phần:

$$\lambda(\|p_u\|_2^2 + \|q_i\|_2^2 + \|q_j\|_2^2)$$

là chuẩn hóa L_2 , trong đó λ kiểm soát mức độ phạt đối với các vector có giá trị quá lớn. BPR Loss có vai trò tối ưu thứ hạng, giúp tăng điểm của Item tích cực, giảm điểm của Item âm và hướng mô hình đến điều kiện:

$$\hat{y}_{ui} > \hat{y}_{uj}$$

Sau khi huấn luyện, các Item chưa tương tác được sắp xếp theo điểm $\hat{y}_{ui}$ giảm dần để lựa chọn danh sách gợi ý Top-K.

e. Ví dụ bằng số cụ thể

- Giả sử: Kích thước chiều ẩn $d = 2$.
 - Vector *User* u_1 : $p_1 = [0.5, 0.8]$
 - Vector *Item* dương i_1 : $q_1 = [0.9, 0.1]$
 - Vector *Item* âm j_1 : $q_2 = [0.1, 0.4]$

- Tính điểm dự đoán:

$$\hat{y}_{u_1, i_1} = p_1^T q_1 = (0.5 \times 0.9) + (0.8 \times 0.1) = 0.45 + 0.08 = 0.53$$

$$\hat{y}_{u_1, j_1} = p_1^T q_2 = (0.5 \times 0.1) + (0.8 \times 0.4) = 0.05 + 0.32 = 0.37$$

- Tính BPR Loss cho bộ ba (u_1, i_1, j_1) :

- Độ chênh lệch điểm: $\Delta = \hat{y}_{u1,i1} - \hat{y}_{u1,j1} = 0.53 - 0.37 = 0.16$
- Giá trị Sigmoid: $\sigma(0.16) = \frac{1}{1+e^{-0.16}} \approx 0.5398$
- Mất mát BPR Loss (chưa tính L2): $\mathcal{L}_{BPR} = -\ln(0.5398) \approx 0.6166$
- Quá trình lan truyền ngược sẽ cập nhật các trọng số trong p_1, q_1, q_2 để đẩy $\hat{y}_{u1,i1}$ lên cao hơn nữa so với $\hat{y}_{u1,j1}$.

3.3. Phương pháp Lai (Hybrid Systems — Late Fusion)

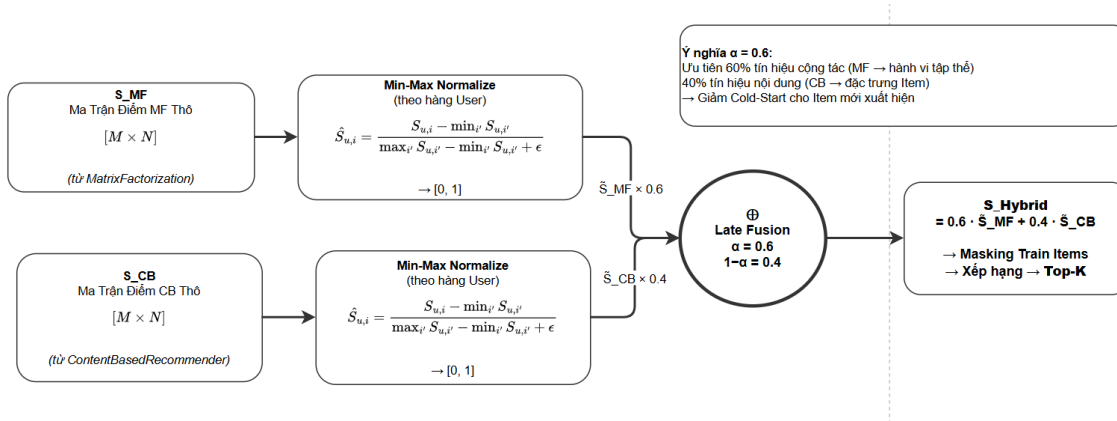
a. Ý tưởng bản chất

Phương pháp Lai kết hợp thế mạnh của cả hai nguồn tín hiệu: thông tin hành vi cộng đồng từ Matrix Factorization và thông tin thuộc tính nội dung từ Content-Based Filtering theo cơ chế Late Fusion (Kết hợp muộn). Phương pháp này giúp khắc phục nhược điểm "Khởi động lạnh" (Cold-start) của CF và nâng cao độ đa dạng gợi ý của CB.

b. Dữ liệu Đầu vào / Đầu ra

- Đầu vào (Input): Ma trận điểm dự đoán độc lập S_{MF} từ mô hình Matrix Factorization và Ma trận điểm Cosine S_{CB} từ mô hình Content-Based.
- Đầu ra (Output): Ma trận điểm tổng hợp S_{Hybrid} dùng để xếp hạng $Top - K$.

c. Bản chất quá trình xử lý và Sơ đồ luồng



Hình 3: Sơ Đồ Kết Hợp Late Fusion (Hybrid: MF + CB)

Giải thích chi tiết các bước trên sơ đồ

- **Chấm điểm độc lập bằng hai mô hình MF và CB:**
 Trước tiên, hệ thống cho hai mô hình Matrix Factorization và Content-Based hoạt động độc lập trên cùng tập dữ liệu người dùng–Item. Mô hình MF khai thác lịch sử tương tác để học các vector đặc trưng ẩn của người dùng và Item, sau đó tính điểm dự đoán bằng

tích vô hướng giữa hai vector. Kết quả thu được là ma trận điểm S_{MF} , trong đó mỗi phần tử $S_{MF}(u, i)$ thể hiện mức độ quan tâm dự đoán của người dùng u đối với Item i dựa trên hành vi tương tác cộng tác. Trong khi đó, mô hình CB sử dụng nội dung của Item, chẳng hạn như tên, mô tả, danh mục hoặc từ khóa, để xây dựng vector TF-IDF. Hồ sơ người dùng được tạo từ các Item đã tương tác, sau đó hệ thống tính Cosine Similarity giữa hồ sơ người dùng và từng Item để tạo ra ma trận điểm S_{CB} . Vì hai mô hình hoạt động độc lập, mỗi mô hình cung cấp một nguồn thông tin khác nhau: MF phản ánh xu hướng hành vi chung của cộng đồng người dùng, còn CB phản ánh sự phù hợp giữa nội dung Item và sở thích cá nhân. Việc duy trì hai nhánh độc lập giúp hệ thống tận dụng đồng thời cả thông tin tương tác và thông tin nội dung trước khi thực hiện bước kết hợp.

- **Chuẩn hóa điểm số bằng Min–Max theo từng người dùng:**

Điểm do MF và CB tạo ra không nằm trên cùng một thang đo. Điểm MF được tính bằng tích vô hướng nên có thể nhận giá trị âm, dương hoặc có biên độ lớn tùy thuộc vào độ lớn của các vector ẩn. Ngược lại, điểm CB được tính bằng Cosine Similarity nên thường nằm trong khoảng từ 0 đến 1 đối với các vector TF-IDF không âm. Nếu cộng trực tiếp hai loại điểm này, mô hình có biên độ lớn hơn có thể chi phối kết quả, dù trọng số kết hợp đã được thiết lập. Vì vậy, hệ thống chuẩn hóa riêng từng ma trận điểm bằng Min–Max Scaler theo từng hàng người dùng:

$$\tilde{S}_{u,i} = \frac{S_{u,i} - \min_{i'} S_{u,i'}}{\max_{i'} S_{u,i'} - \min_{i'} S_{u,i'} + \varepsilon}$$

Trong đó, $S_{u,i}$ là điểm ban đầu của người dùng u đối với Item i ; $\min_{i'} S_{u,i'}$ và $\max_{i'} S_{u,i'}$ lần lượt là điểm nhỏ nhất và lớn nhất trong toàn bộ Item của người dùng đó. Giá trị ε là một số dương rất nhỏ, thường đặt bằng 10^{-8} , được thêm vào mẫu số để tránh lỗi chia cho 0 khi tất cả Item có cùng điểm. Sau chuẩn hóa, điểm thấp nhất của mỗi người dùng được đưa về gần 0, còn điểm cao nhất được đưa về gần 1. Quá trình này được thực hiện độc lập cho S_{MF} và S_{CB} , tạo ra hai ma trận chuẩn hóa $\tilde{S}_{MF}$ và $\tilde{S}_{CB}$. Chuẩn hóa theo từng người dùng giúp giữ lại thứ tự ưu tiên tương đối giữa các Item của chính người dùng đó, đồng thời đưa hai nguồn điểm về cùng thang đo $[0, 1]$ để có thể kết hợp một cách công bằng.

- **Kết hợp điểm bằng phương pháp Late Fusion:**

Sau khi hai ma trận điểm đã được chuẩn hóa, hệ thống thực hiện Late Fusion, tức là kết hợp ở giai đoạn điểm dự đoán cuối cùng thay vì trộn dữ liệu hoặc đặc trưng từ đầu. Điểm Hybrid của người dùng u đối với Item i được tính bằng phép cộng có trọng số:

$$S_{\text{Hybrid}}(u, i) = \alpha \tilde{S}_{MF}(u, i) + (1 - \alpha) \tilde{S}_{CB}(u, i)$$

Trong đó, $\alpha \in [0, 1]$ là tham số điều chỉnh mức đóng góp của hai mô hình. Thành phần $\alpha \tilde{S}_{MF}(u, i)$ biểu diễn đóng góp của tín hiệu hành vi, còn thành phần $(1 - \alpha) \tilde{S}_{CB}(u, i)$ biểu diễn đóng góp của tín hiệu nội dung. Khi $\alpha > 0.5$, hệ thống ưu tiên mô hình MF, phù hợp trong trường hợp dữ liệu tương tác đã đủ lớn và có thể phản ánh tốt xu hướng hành vi của người dùng. Khi $\alpha < 0.5$, mô hình CB được ưu tiên hơn, phù hợp khi thông tin nội dung Item đầy đủ hoặc hệ thống cần hỗ trợ các Item chưa có nhiều tương tác. Nếu $\alpha = 0.5$, hai mô hình đóng góp ngang nhau. Ví dụ, khi $\alpha = 0.6$, điểm cuối cùng gồm 60% điểm MF và 40% điểm CB. Giá trị α có thể được lựa chọn thông qua thực nghiệm trên tập validation để đạt chất lượng gợi ý tốt nhất.

d. Công thức toán học

- Chuẩn hóa Min-Max (theo từng hàng *User* u):

$$\hat{S}_{u,i} = \frac{S_{u,i} - \min_{i'} S_{u,i'}}{\max_{i'} S_{u,i'} - \min_{i'} S_{u,i'} + \epsilon}$$

Trong đó: $S_{u,i}$ là điểm dự đoán ban đầu của người dùng u đối với Item i ; $\hat{S}_{u,i}$ là điểm sau khi chuẩn hóa; i' đại diện cho toàn bộ các Item được xét của người dùng u . Hai đại lượng $\min_{i'} S_{u,i'}$ và $\max_{i'} S_{u,i'}$ lần lượt là điểm nhỏ nhất và điểm lớn nhất trên hàng tương ứng với người dùng đó. Giá trị $\epsilon = 10^{-8}$ được thêm vào mẫu số để tránh lỗi chia cho 0 khi tất cả Item có cùng điểm.

Công thức này có vai trò đưa điểm dự đoán của từng mô hình về cùng thang đo gần $[0, 1]$. Việc chuẩn hóa được thực hiện riêng theo từng người dùng, giúp giữ nguyên thứ tự ưu tiên tương đối giữa các Item, đồng thời tránh trường hợp mô hình có biên độ điểm lớn hơn chi phối kết quả kết hợp.

- Công thức Late Fusion:

$$S_{Hybrid}(u, i) = \alpha \cdot \hat{S}_{MF}(u, i) + (1 - \alpha) \cdot \hat{S}_{CB}(u, i)$$

Trong đó: $S_{Hybrid}(u, i)$ là điểm gợi ý cuối cùng của hệ thống Hybrid; $\hat{S}_{MF}(u, i)$ là điểm đã chuẩn hóa từ mô hình Matrix Factorization; $\hat{S}_{CB}(u, i)$ là điểm đã chuẩn hóa từ mô hình Content-Based. Tham số $\alpha \in [0, 1]$ xác định mức đóng góp của MF, còn $1 - \alpha$ xác định mức đóng góp của CB.

Công thức Late Fusion có vai trò kết hợp tín hiệu hành vi từ MF với tín hiệu nội dung từ CB. Khi $\alpha > 0.5$, hệ thống ưu tiên nhiều hơn cho mô hình MF; khi $\alpha < 0.5$, hệ thống ưu tiên mô hình CB. Sau khi tính điểm Hybrid cho các Item chưa tương tác, hệ thống sắp xếp điểm giảm dần và chọn K Item có điểm cao nhất để tạo danh sách gợi ý Top- K .

e. Ví dụ bằng số cụ thể

- Giả sử: Xét *User* u_1 đối với *Item* i_1 .
 - Mô hình MF cho điểm $S_{MF}(u_1, i_1) = 3.2$ (với điểm nhỏ nhất của u_1 là 1.0, lớn nhất là 5.0).
 - Mô hình CB cho điểm $S_{CB}(u_1, i_1) = 0.8$ (với điểm nhỏ nhất của u_1 là 0.2, lớn nhất là 1.0).

- Step 1: Chuẩn hóa Min-Max về $[0,1]$:

$$\hat{S}_{MF}(u_1, i_1) = \frac{3.2 - 1.0}{5.0 - 1.0} = \frac{2.2}{4.0} = 0.55$$

$$\hat{S}_{CB}(u_1, i_1) = \frac{0.8 - 0.2}{1.0 - 0.2} = \frac{0.6}{0.8} = 0.75$$

- Step 2: Tính điểm Hybrid với $\alpha = 0.6$:

$$S_{Hybrid}(u_1, i_1) = 0.6 \times 0.55 + (1 - 0.6) \times 0.75 = 0.33 + 0.30 = 0.63$$

3.4. Mạng nơ-ron đồ thị (LightGCN — Đồ thị lưỡng phân)

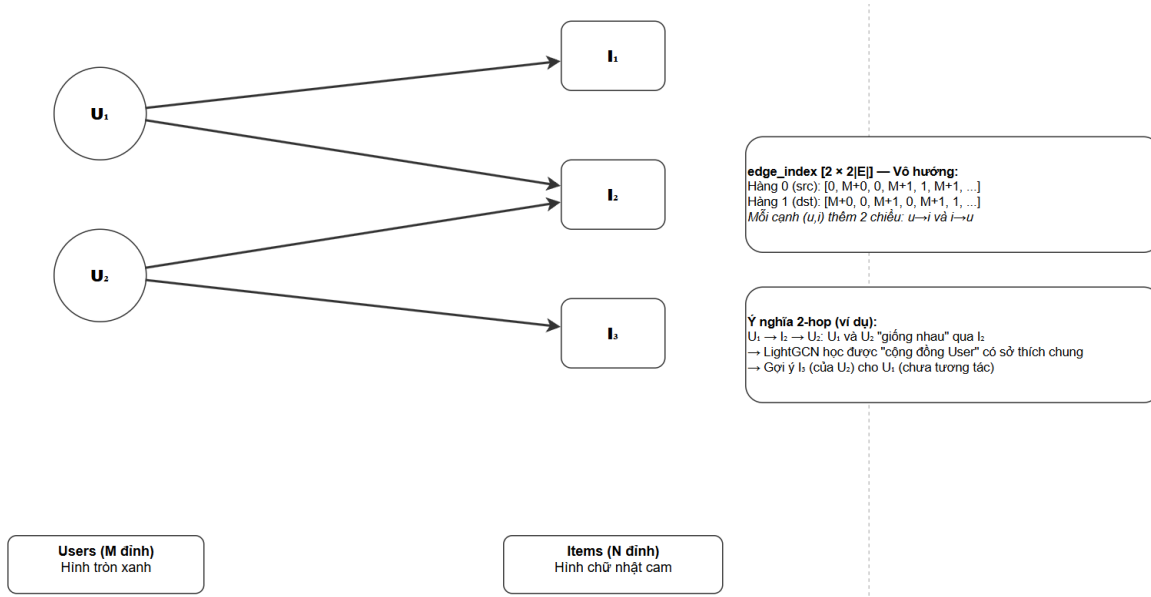
a. Ý tưởng bản chất

LightGCN biểu diễn toàn bộ bài toán tư vấn dưới dạng một Đồ thị lưỡng phân User–Item (Bipartite Graph) $G = (U \cup I, E)$. Tận dụng cơ chế lan truyền thông điệp (*Message Passing*), LightGCN thu thập thông tin láng giềng qua nhiều tầng (K -hops) để học quan hệ cấu trúc bậc cao (User $\rightarrow$ Item $\rightarrow$ User). Kiến trúc được thiết kế tối giản, loại bỏ hoàn toàn ma trận trọng số và hàm kích hoạt phi tuyến để tối ưu hiệu năng cho bài toán xếp hạng Top-K.

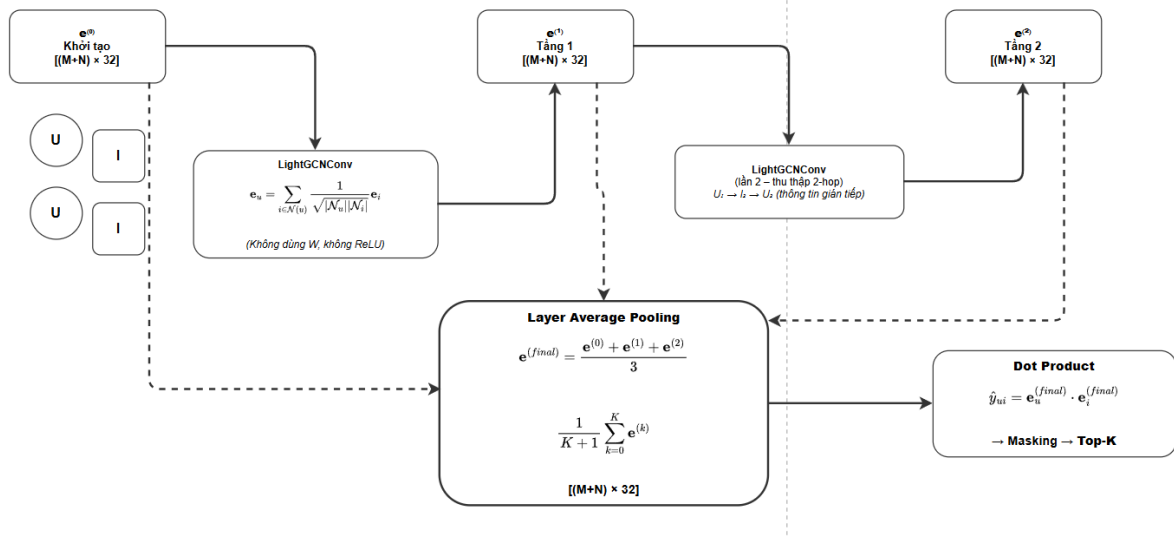
b. Dữ liệu Đầu vào / Đầu ra

- Đầu vào (Input): Đồ thị lưỡng phân $G = (U \cup I, E)$ biểu diễn qua danh sách cạnh tương tác `edge_index`.
- Đầu ra (Output): Vector nhúng biểu diễn đa tầng $e_u^{(final)}, e_i^{(final)}$ và điểm dự đoán $\hat{y}_{ui}$.

c. Bản chất quá trình xử lý và Sơ đồ luồng



Hình 4 : Đồ Thị Lưỡng Phân User–Item



Hình 5 : Sơ Đồ Lan Truyền Thông Điệp

Giải thích chi tiết các bước trên sơ đồ

- Biểu diễn dữ liệu dưới dạng đồ thị lưỡng phân:**

LightGCN mô hình hóa dữ liệu tương tác dưới dạng một đồ thị lưỡng phân gồm hai tập đỉnh riêng biệt: tập người dùng và tập Item. Mỗi người dùng u được biểu diễn bởi một đỉnh User, còn mỗi Item i được biểu diễn bởi một đỉnh Item. Khi người dùng u đã tương tác với Item i , tức là $r_{ui} = 1$, hệ thống tạo một cạnh nối giữa hai đỉnh này. Tương tác có thể là xem, mua, yêu thích, đánh giá hoặc lựa chọn Item. Do đồ thị là lưỡng phân nên không tồn tại cạnh trực tiếp giữa hai User hoặc giữa hai Item. Các mối quan hệ tương

đồng được học gián tiếp thông qua những Item hoặc User chung. Ví dụ, hai người dùng cùng tương tác với nhiều Item giống nhau có thể được xem là có sở thích tương tự, còn hai Item được nhiều người dùng giống nhau lựa chọn có thể mang đặc điểm cộng tác gần nhau.

- **Lan truyền thông điệp giữa các đỉnh láng giềng:**

Ban đầu, mỗi người dùng u và Item i được gán một vector Embedding lần lượt là $e_u^{(0)}$ và $e_i^{(0)}$. Qua mỗi lớp lan truyền k , một đỉnh User thu thập thông tin từ tất cả các Item thuộc tập láng giềng $\mathcal{N}_u$, còn một đỉnh Item thu thập thông tin từ các User thuộc tập $\mathcal{N}_i$. Embedding của người dùng tại lớp k được hình thành từ Embedding lớp $k - 1$ của các Item đã tương tác với người dùng đó. Tương tự, Embedding của Item được cập nhật từ các User đã tương tác với Item. Quá trình này giúp thông tin sở thích và đặc điểm Item được truyền qua các cạnh của đồ thị. LightGCN chỉ thực hiện phép tổng hợp tuyến tính từ các nút láng giềng, không sử dụng ma trận biến đổi đặc trưng hoặc hàm kích hoạt phi tuyến phức tạp, nhờ đó tập trung trực tiếp vào tín hiệu cộng tác trong cấu trúc đồ thị.

- **Chuẩn hóa thông tin theo bậc của đỉnh:**

Khi tổng hợp thông tin từ một cạnh nối giữa User u và Item i , mô hình sử dụng hệ số chuẩn hóa:

$$\frac{1}{\sqrt{|\mathcal{N}_u|} \sqrt{|\mathcal{N}_i|}}$$

Trong đó, $|\mathcal{N}_u|$ là số Item mà người dùng u đã tương tác, còn $|\mathcal{N}_i|$ là số người dùng đã tương tác với Item i . Hệ số này giúp giảm ảnh hưởng quá lớn của các đỉnh có nhiều kết nối. Nếu không chuẩn hóa, những User hoạt động nhiều hoặc những Item rất phổ biến có thể đóng góp quá mạnh vào quá trình cập nhật Embedding, làm cho các vector bị lệch về phía các đỉnh phổ biến. Nhờ phép chuẩn hóa theo căn bậc hai của tích hai bậc đỉnh, thông tin từ từng cạnh được điều chỉnh cân bằng hơn, giúp mô hình học ổn định và phản ánh tốt hơn mối quan hệ cộng tác giữa User và Item.

- **Tổng hợp Embedding qua nhiều tầng:**

Sau mỗi lớp lan truyền, mô hình thu được một biểu diễn mới cho từng User và Item. Embedding ở lớp thấp phản ánh các quan hệ tương tác trực tiếp, trong khi Embedding ở lớp cao hơn chứa thông tin cộng tác gián tiếp từ những nút cách xa hơn trên đồ thị. Tuy nhiên, nếu chỉ sử dụng Embedding của lớp cuối cùng hoặc lan truyền qua quá nhiều lớp, các vector có thể trở nên quá giống nhau, làm mất đặc trưng riêng của từng đỉnh. Hiện tượng này được gọi là Over-smoothing. Để hạn chế vấn đề đó, LightGCN kết hợp Embedding từ tất cả các lớp, bao gồm cả lớp khởi tạo 0, bằng phép trung bình:

$$e_u^{(\text{final})} = \frac{1}{K+1} \sum_{k=0}^K e_u^{(k)}$$

$$e_i^{(\text{final})} = \frac{1}{K+1} \sum_{k=0}^K e_i^{(k)}$$

Cách tổng hợp này giúp giữ lại cả thông tin ban đầu, quan hệ trực tiếp và các quan hệ bậc cao, từ đó tạo ra Embedding cuối cùng đầy đủ hơn cho người dùng và Item.

- **Tính điểm dự đoán bằng Dot Product:**

Sau khi thu được Embedding cuối cùng, mức độ phù hợp giữa người dùng u và Item i được tính bằng tích vô hướng:

$$\hat{y}_{ui} = \left(e_u^{(\text{final})} \right)^T e_i^{(\text{final})}$$

Nếu hai vector có hướng và các thành phần tương đồng, giá trị $\hat{y}_{ui}$ sẽ lớn, cho thấy Item i có khả năng phù hợp với sở thích của người dùng u . Ngược lại, điểm thấp biểu thị mức độ phù hợp thấp hơn. Hệ thống tính điểm cho các Item mà người dùng chưa tương tác, loại bỏ các Item đã xuất hiện trong lịch sử, sắp xếp điểm theo thứ tự giảm dần và chọn K Item có điểm cao nhất để tạo danh sách gợi ý Top- K .

d. Công thức toán học

- Khởi tạo (Lớp 0):

$$e_u^{(0)} = \text{Embedding}(u); \quad e_i^{(0)} = \text{Embedding}(i)$$

Trong đó: $e_u^{(0)}$ là vector Embedding ban đầu của người dùng u , còn $e_i^{(0)}$ là vector Embedding ban đầu của Item i . Chỉ số 0 thể hiện đây là lớp khởi tạo, khi các vector chưa tiếp nhận thông tin từ những nút láng giềng trên đồ thị.

Công thức này có vai trò tạo biểu diễn ban đầu cho mỗi User và Item trong không gian đặc trưng ẩn. Các vector thường được khởi tạo bằng những giá trị ngẫu nhiên nhỏ, sau đó được cập nhật trong quá trình huấn luyện để phản ánh sở thích của người dùng và đặc điểm cộng tác của Item.

- Lan truyền Message Passing (Lớp k):

$$e_u^{(k)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}} e_i^{(k-1)}$$

$$e_i^{(k)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i| |\mathcal{N}_u|}} e_u^{(k-1)}$$

Trong đó: $e_u^{(k)}$ và $e_i^{(k)}$ lần lượt là Embedding của người dùng u và Item i tại lớp k . $\mathcal{N}_u$ là tập các Item đã kết nối với người dùng u , còn $\mathcal{N}_i$ là tập các User đã kết nối với Item i . Các vector $e_i^{(k-1)}$ và $e_u^{(k-1)}$ là thông tin được truyền từ lớp trước.

Hệ số: $\frac{1}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}}$ dùng để chuẩn hóa mức đóng góp của mỗi nút láng giềng theo số lượng kết nối. Công thức Message Passing có vai trò giúp User thu thập thông tin từ các Item đã tương tác và Item thu thập thông tin từ các User liên quan. Nhờ lan truyền qua nhiều lớp, mô hình có thể học được cả quan hệ trực tiếp và quan hệ cộng tác gián tiếp trên đồ thị.

- Tổng hợp Embedding đa tầng (Layer Combination):

$$e_u^{(final)} = \frac{1}{K+1} \sum_{k=0}^K e_u^{(k)}; \quad e_i^{(final)} = \frac{1}{K+1} \sum_{k=0}^K e_i^{(k)}$$

Trong đó: K là số lớp lan truyền thông tin; $e_u^{(k)}$ và $e_i^{(k)}$ là Embedding của User và Item tại lớp k . Tổng được tính từ lớp khởi tạo $k = 0$ đến lớp cuối cùng $k = K$, còn hệ số $\frac{1}{K+1}$ thực hiện phép lấy trung bình của toàn bộ $K + 1$ lớp.

Công thức này có vai trò kết hợp thông tin ở nhiều mức khác nhau. Embedding lớp thấp chứa nhiều thông tin tương tác trực tiếp, trong khi các lớp cao phản ánh quan hệ cộng tác xa hơn. Việc lấy trung bình các lớp giúp giữ lại thông tin ban đầu, khai thác quan hệ bậc cao và hạn chế hiện tượng các vector trở nên quá giống nhau khi lan truyền quá sâu.

- Dự đoán điểm xếp hạng:

$$\hat{y}_{ui} = e_u^{(final)T} e_i^{(final)}$$

Trong đó: $\hat{y}_{ui}$ là điểm dự đoán mức độ phù hợp giữa người dùng u và Item i ; $e_u^{(final)}$ là Embedding cuối cùng của người dùng; $e_i^{(final)}$ là Embedding cuối cùng của Item. Ký hiệu T biểu thị phép chuyển vị để thực hiện tích vô hướng giữa hai vector.

Công thức này có vai trò đo mức độ tương đồng giữa biểu diễn cuối cùng của User và Item. Điểm $\hat{y}_{ui}$ càng lớn thì Item i càng được dự đoán phù hợp với sở thích của người dùng u . Sau khi tính điểm cho các Item chưa tương tác, hệ thống sắp xếp chúng theo thứ tự giảm dần và chọn K Item có điểm cao nhất để tạo danh sách gợi ý Top- K .

e. Ví dụ bằng số cụ thể

- Giả sử cấu trúc đồ thị: *User* u_1 tương tác với 2 *Item* i_1 và i_2 .

- Bậc láng giềng: $|\mathcal{N}_{u1}| = 2$, $|\mathcal{N}_{i1}| = 1$, $|\mathcal{N}_{i2}| = 1$.
- Embedding khởi tạo ($d = 2$):

- $e_{u1}^{(0)} = [0.1, 0.3]$

- $e_{i1}^{(0)} = [0.4, 0.2]$

- $e_{i2}^{(0)} = [0.6, 0.8]$

- Tính lan truyền Lớp 1 ($k = 1$) cho *User* u_1 :

- Hệ số chuẩn hóa từ $i_1 \rightarrow u_1$: $\frac{1}{\sqrt{|\mathcal{N}_{u1}|}|\mathcal{N}_{i1}|}} = \frac{1}{\sqrt{2 \times 1}} \approx 0.7071$

- Hệ số chuẩn hóa từ $i_2 \rightarrow u_1$: $\frac{1}{\sqrt{|\mathcal{N}_{u1}|}|\mathcal{N}_{i2}|}} = \frac{1}{\sqrt{2 \times 1}} \approx 0.7071$

- Vector $e_{u1}^{(1)}$ sau lan truyền:

$$e_{u1}^{(1)} = 0.7071 \times [0.4, 0.2] + 0.7071 \times [0.6, 0.8] = [0.2828, 0.1414] + [0.4243, 0.5657] \\ = [0.7071, 0.7071]$$

- Tính Embedding tổng hợp cuối cùng ($K = 1$ tầng):

$$e_{u1}^{(final)} = \frac{e_{u1}^{(0)} + e_{u1}^{(1)}}{2} = \frac{[0.1, 0.3] + [0.7071, 0.7071]}{2} = \frac{[0.8071, 1.0071]}{2} = [0.4036, 0.50365]$$

- Vector $e_{u1}^{(final)}$ thu được đã tổng hợp hài hòa giữa đặc trưng sở thích ban đầu và thông tin cấu trúc mạng từ các sản phẩm đã tương tác.

CHƯƠNG IV: CÀI ĐẶT HỆ THỐNG

4.1. Kiến trúc hệ thống tổng thể

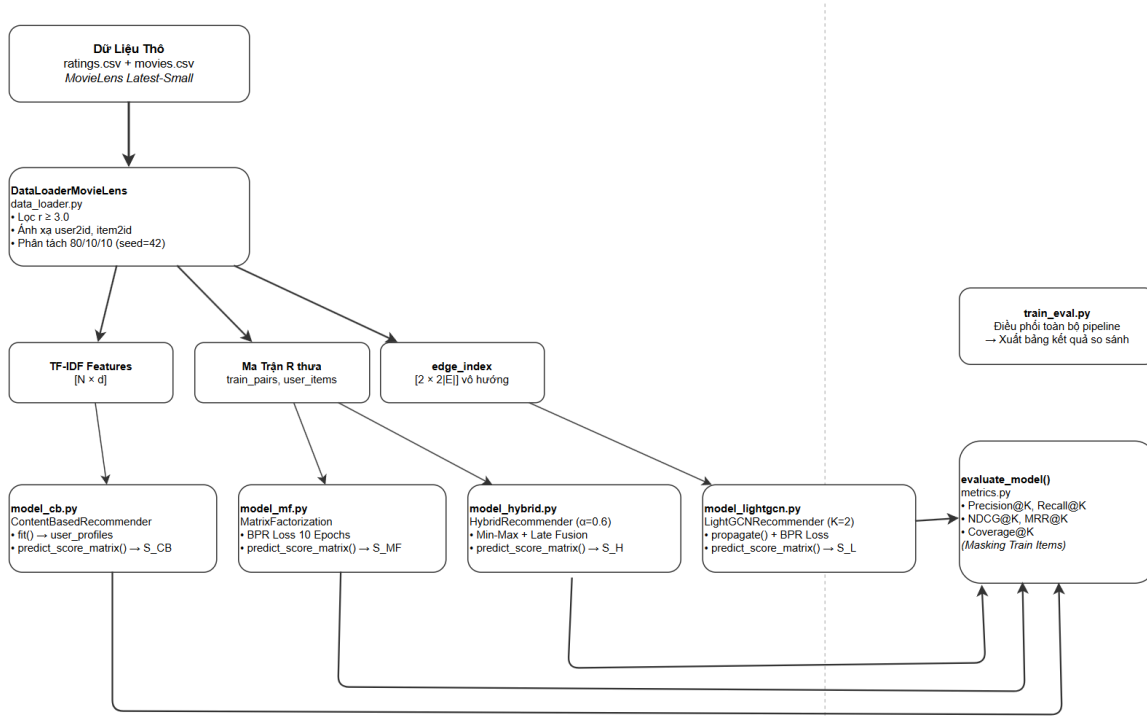
Toàn bộ hệ thống tư vấn được tổ chức thành 4 khối chức năng chính tương ứng với 7 tệp mã nguồn chuyên biệt trong thư mục src/:

Khối 1 - Tiền xử lý dữ liệu & Xây dựng Đồ thị (data_loader.py): Tiếp nhận dữ liệu thô, thực hiện mã hóa định danh hai chiều (user2id, item2id), trích xuất ma trận đặc trưng thuộc tính TF-IDF (V_{Item}), phân tách tập dữ liệu (Train/Val/Test theo tỷ lệ 80/10/10) và đóng gói ma trận kề Đồ thị Lưỡng phân (edge_index).

Khối 2 - Các Mô hình Tư vấn thành phần (model_cb.py, model_mf.py, model_hybrid.py, model_lightgcn.py): Hiện thực hóa lý thuyết toán học của 4 thuật toán thành các Class mô hình, nhận dữ liệu đầu vào đã chuẩn hóa và tính toán ma trận điểm số dự đoán $S \in \mathbb{R}^{M \times N}$.

Khối 3 - Thư viện Độ đo Đánh giá (metrics.py): Đóng gói các công thức toán học đánh giá chất lượng xếp hạng Top-K kết hợp cơ chế che sản phẩm đã tương tác trong tập huấn luyện (Train Masking).

Khối 4 - Điều phối Huấn luyện & Đánh giá (train_eval.py): Kịch bản trung tâm quản lý vòng lặp huấn luyện, lấy mẫu âm, tối ưu BPR Loss và gọi bộ đo để xuất kết quả thực nghiệm.



4.2. Luồng thực thi phương pháp Lọc nội dung

Phương pháp Lọc nội dung được cài đặt thông qua Class **ContentBasedRecommender** (model_cb.py), phối hợp cùng phương thức trích xuất đặc trưng **build_tfidf_features()** (data_loader.py). Quá trình hiện thực hóa mô hình bao gồm 3 bước với các điểm ánh xạ lý thuyết và kỹ thuật tối ưu hóa mã nguồn như sau:

Bước 1: Vector hóa thuộc tính và Chuẩn hóa L_2 cho đặc trưng Item

- Phương thức **build_tfidf_features()** tiếp nhận danh mục N sản phẩm, duyệt qua chuỗi thuộc tính thể loại để tính toán tần suất xuất hiện TF và độ hiếm IDF của từng từ khóa theo công thức:

$$W(t, d) = TF(t, d) \times \log\left(\frac{N}{DF(t)}\right)$$

- Kết quả tính toán được đóng gói thành ma trận đặc trưng $V_{Item} \in \mathbb{R}^{N \times d_{features}}$. Ngay sau đó, chương trình tiến hành chuẩn hóa độ dài Euclidean (L_2 norm) cho từng vector đặc trưng sản phẩm V_i :

$$\tilde{V}_i = \frac{V_i}{\|V_i\|_2} = \frac{V_i}{\sqrt{\sum_{k=1}^d V_{i,k}^2}}$$

Việc chuẩn hóa trước cho tập *Item* đảm bảo mỗi vector đặc trưng $\tilde{V}_i$ luôn có độ dài chuẩn bằng 1 ($\|\tilde{V}_i\|_2 = 1$), giúp tiết kiệm thời gian tính toán mẫu số nhiều lần ở các giai đoạn sau.

Bước 2: Xây dựng và Chuẩn hóa Hồ sơ Người dùng

- Phương thức **fit()** trong lớp **ContentBasedRecommender** tiếp nhận tập dữ liệu huấn luyện *train_user_items*. Với mỗi người dùng u , hàm tính vector sở thích trung bình U_u dựa trên tổng các vector sản phẩm $\tilde{V}_i$ mà người dùng đó đã từng tương tác ($i \in I_{u+}^{train}$):

$$U_u = \frac{1}{|I_{u+}^{train}|} \sum_{i \in I_{u+}^{train}} \tilde{V}_i$$

- Ngay sau khi tính xong vector trung bình U_u , hàm tiếp tục thực hiện chuẩn hóa L_2 cho chính vector Hồ sơ Người dùng:

$$\tilde{U}_u = \frac{U_u}{\|U_u\|_2} = \frac{U_u}{\sqrt{\sum_{k=1}^d U_{u,k}^2}}$$

Việc đưa độ dài vector $\|\tilde{U}_u\|_2$ về đúng 1 ngay tại thời điểm khởi tạo hồ sơ giúp hoàn tất công đoạn tiền xử lý mẫu số cho cả *User* và *Item*.

Bước 3: Chấm điểm tương đồng bằng Tích vô hướng

- Độ đo Cosine Similarity giữa người dùng u và sản phẩm i có công thức tổng quát:

$$\text{Cosine}(U_u, V_i) = \frac{U_u \cdot V_i}{\|U_u\|_2 \cdot \|V_i\|_2}$$

- Do các vector biểu diễn người dùng ($\tilde{U}_u$) và sản phẩm ($\tilde{V}_i$) đều đã được chuẩn hóa L_2 ở hai bước trên (mẫu số $\|\tilde{U}_u\|_2 \cdot \|\tilde{V}_i\|_2 = 1$), độ đo Cosine Similarity được đơn giản hóa hoàn toàn về phép tính tích vô hướng (Dot Product):

$$\text{Score}(u, i) = \text{Cosine}(U_u, V_i) = \tilde{U}_u \cdot \tilde{V}_i^T = \sum_{k=1}^{d_{\text{features}}} \tilde{U}_{u,k} \times \tilde{V}_{i,k}$$

Phương thức **predict_score_matrix()** trong `model_cb.py` thực hiện vòng lặp nhân tích vô hướng giữa vector $\tilde{U}_u$ và $\tilde{V}_i$, gán kết quả vào ma trận $S_{CB} \in \mathbb{R}^{M \times N}$.

Kết luận: Việc chuyển phép tính Cosine góc thành Tích vô hướng đơn giản nhờ chuẩn hóa L_2 trước giúp hệ thống cắt giảm $M \times N$ phép tính căn bậc hai dư thừa ở thời điểm dự đoán, giảm độ phức tạp tính toán mẫu số từ $O(M \times N)$ xuống $O(M + N)$, vừa đảm bảo tính chính xác 100% theo đúng định nghĩa toán học của Cosine Similarity, vừa giúp hệ thống đạt tốc độ xử lý nhanh nhất.

4.3. Luồng thực thi Phân rã ma trận

Mô hình Phân rã ma trận được cài đặt thông qua class **MatrixFactorization** (kế thừa từ `nn.Module` trong tệp **model_mf.py**), kết hợp chặt chẽ với cơ chế lấy mẫu âm trong kịch bản điều phối **train_eval.py**. Luồng xử lý được hiện thực hóa qua 3 bước với các điểm ánh xạ cụ thể như sau:

Bước 1: Khởi tạo Bảng nhúng không gian ẩn

- Tại phương thức khởi tạo **init()** của lớp **MatrixFactorization**, hệ thống định nghĩa hai bảng nhúng tra cứu đại diện cho ma trận nhúng người dùng $P \in \mathbb{R}^{M \times d}$ và ma trận nhúng sản phẩm $Q \in \mathbb{R}^{N \times d}$ với số chiều ẩn $d = 32$
- Trọng số ban đầu của các vector được khởi tạo ngẫu nhiên theo phân phối chuẩn $N(0,0.1)$ thông qua hàm **nn.init.normal_()**. Cấu trúc **nn.Embedding** của PyTorch biến thao tác truy xuất vector p_u hoặc q_i từ ma trận tổng thể thành phép tra cứu chỉ mục có độ phức tạp $O(1)$, đồng thời tự động ghi nhận Gradient để cập nhật trọng số trong quá trình lan truyền ngược (Backpropagation).

Bước 2: Lấy mẫu âm và Dự đoán điểm tương tác

- Trong từng Epoch huấn luyện, hàm **sample_negative_items()** trong tệp `train_eval.py` duyệt qua từng tương tác dương $(u, i) \in \text{Train}$. Với mỗi cặp tương tác, thuật toán rút ngẫu nhiên một sản phẩm j bất kỳ và kiểm tra điều kiện $j \notin I_{u+}^{\text{train}}$ bằng vòng lặp `while` để đảm bảo j là mẫu tương tác âm hợp lệ đối với người dùng u . Các bộ ba (u, i, j) sau đó được gom thành từng lô dữ liệu (Batch).
- Khi lô dữ liệu được đưa vào mô hình, phương thức **forward()** thực hiện trích xuất các vector nhúng tương ứng và tính điểm số dự đoán $\hat{y}_{ui}$ bằng tích vô hướng giữa vector người dùng p_u và vector sản phẩm q_i :

$$\hat{y}_{ui} = p_u^T q_i = \sum_{k=1}^d p_{u,k} \times q_{i,k}$$

- Trong mã nguồn, phép tính này được tối ưu hóa bằng phép nhân song song từng phần tử trên Tensor và tính tổng theo chiều ẩn: `(u_embed * i_embed).sum(dim=-1)`.

Bước 3: Tối ưu hóa Hàm mất mát BPR và Ma trận dự đoán

- Phương thức **compute_bpr_loss()** tiếp nhận các lô bộ ba (u, i, j) để tính toán hàm mất mát xếp hạng BPR (Bayesian Personalized Ranking) kết hợp phạt chuẩn hóa L_2 :

$$\mathcal{L}_{BPR} = - \sum_{(u,i,j)} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda(\|p_u\|^2 + \|q_i\|^2 + \|q_j\|^2)$$

- Trong phương thức, độ chênh lệch điểm số **pos_scores - neg_scores** ($\hat{y}_{ui} - \hat{y}_{uj}$) được đưa qua hàm kích hoạt **torch.sigmoid()** và lấy logarit âm trung bình - **torch.mean(torch.log(...))** để thu được *bpr_loss*. Khoản phạt chuẩn hóa L_2 (**reg_loss**) được tính bằng tổng bình phương độ dài của các vector nhúng nhân với hệ số $\lambda = 10^{-4}$. Tổng của hai giá trị này tạo thành tín hiệu mất mát giúp bộ tối ưu Adam cập nhật bảng nhúng.
- Tại giai đoạn dự đoán để xuất danh sách Top-K, phương thức **predict_score_matrix()** trích xuất toàn bộ ma trận trọng số người dùng P (**self.user_embedding.weight**) và ma trận sản phẩm Q (**self.item_embedding.weight**), sau đó thực hiện phép nhân ma trận toàn cục:

$$\mathbf{S}_{MF} = P \times Q^T$$

Thao tác **torch.matmul(P, Q.t())** tận dụng kiến trúc tính toán song song trên GPU, tính toán toàn bộ ma trận điểm số dự đoán $\mathbf{S}_{MF} \in \mathbb{R}^{M \times N}$ cho M người dùng và N sản phẩm chỉ trong một vài lệnh, giúp tăng tốc độ dự đoán toàn hệ thống lên gấp nhiều lần so với việc duyệt mảng tính toán tuần tự.

4.4. Luồng thực thi Phương pháp Lai

Mô hình Lai ứng dụng chiến lược Kết hợp muộn (Late Fusion) được cài đặt trọn vẹn trong class **HybridRecommender** thuộc tệp **model_hybrid.py**. Mô hình đóng vai trò là mô-đun tích hợp kết quả dự đoán từ hai hướng tiếp cận độc lập: Lọc cộng tác Phân rã ma trận (MF) và Lọc nội dung (CB), nhằm khai thác tối đa ưu điểm hành vi cộng đồng kết hợp thông tin thuộc tính sản phẩm. Luồng thực thi gồm 3 bước với các điểm ánh xạ lý thuyết và kỹ thuật cài đặt cụ thể như sau:

Bước 1: Tiếp nhận ma trận điểm số thành phần

- Đầu vào của mô hình Lai là hai ma trận điểm dự đoán độc lập $\mathbf{S}_{MF} \in \mathbb{R}^{M \times N}$ (từ Phân rã ma trận) và $\mathbf{S}_{CB} \in \mathbb{R}^{M \times N}$ (từ Lọc nội dung) như đã trình bày ở Mục 3.3 ở Chương III.
- Phương thức **predict_score_matrix(score_matrix_mf, score_matrix_cb)** trong class **HybridRecommender** tiếp nhận trực tiếp hai ma trận điểm thô sau khi hai mô hình MF và CB đã hoàn tất giai đoạn tính toán riêng biệt. Class không chứa các tham số học qua Gradient mà đóng vai trò là một bộ điều phối đại số tuyến tính trên ma trận.

Bước 2: Chuẩn hóa Min-Max theo từng người dùng

- Phương thức **user_wise_min_max_scale()** thực hiện duyệt qua từng hàng tương ứng với mỗi người dùng u trên ma trận điểm. Với mỗi hàng, thuật toán tìm giá trị nhỏ nhất (**min_val**) và lớn nhất (**max_val**) của riêng người dùng đó, sau đó tính khoảng chênh lệch $\text{diff} = \text{max_val} - \text{min_val}$. Mỗi phần tử điểm $S_{u,i}$ được biến đổi về khoảng $[0,1]$ theo công thức:

$$\hat{S}_{u,i} = \frac{S_{u,i} - \min_{i'} S_{u,i'}}{\max_{i'} S_{u,i'} - \min_{i'} S_{u,i'} + \epsilon}$$

- Trong mã nguồn, quá trình biến đổi thực tế được tính bằng tỷ lệ:

$$\hat{S}_{u,i} = \frac{S_{u,i} - \text{min_val}}{\text{diff}}$$

Hàm cũng cài đặt thêm điều kiện kiểm tra an toàn **if diff < 1e-8** để gán toàn bộ hàng bằng 0.0 nếu tất cả điểm số của người dùng đó bằng nhau, giúp tránh triệt để lỗi chia cho số 0 (**ZeroDivisionError**).

- Do điểm $\mathbf{S}_{MF}$ thu được từ tích vô hướng (không giới hạn biên độ) còn điểm $\mathbf{S}_{CB}$ thu được từ Cosine Similarity (biên độ $[-1,1]$ hoặc $[0,1]$), việc chuẩn hóa Min-Max theo từng hàng *User* đưa hai dải điểm về cùng một thang đo $[0,1]$ chuẩn mực. Kỹ thuật chuẩn hóa theo hàng (*User-wise*) bảo toàn chính xác 100% thứ tự ưu tiên xếp hạng cá nhân của từng người dùng mà không bị lệch do phân bố điểm số chung của toàn hệ thống.

Bước 3: Tích hợp Kết hợp muộn có trọng số

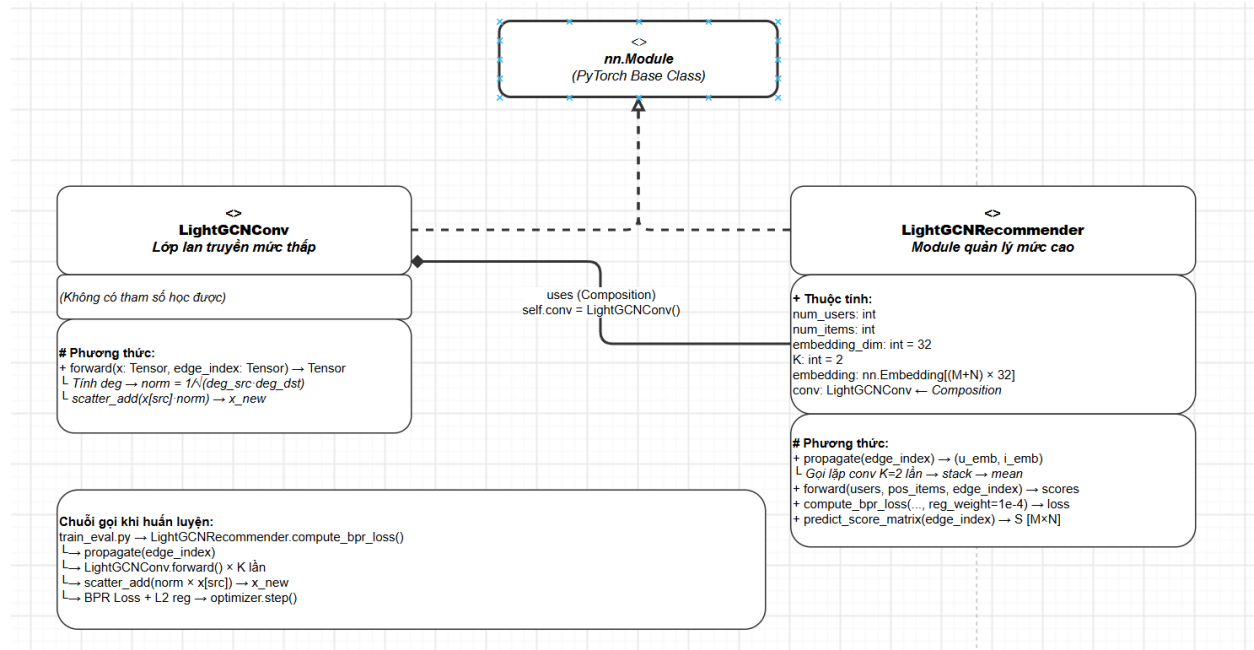
- Sau khi chuẩn hóa, phương thức **predict_score_matrix()** tiến hành cộng tổ hợp tuyến tính có trọng số hai ma trận điểm $\hat{\mathbf{S}}_{MF}$ và $\hat{\mathbf{S}}_{CB}$ theo công thức:

$$\mathbf{S}_{Hybrid}(u, i) = \alpha \cdot \hat{\mathbf{S}}_{MF}(u, i) + (1 - \alpha) \cdot \hat{\mathbf{S}}_{CB}(u, i)$$

- Tham số trọng số α được khởi tạo mặc định bằng 0.6 tại phương thức **init()**. Việc thiết lập $\alpha = 0.6$ nhằm dành 60% trọng số cho nhánh Phân rã ma trận để ưu tiên khả năng khám phá xu hướng hành vi cộng đồng và đạt độ chính xác cao trên các sản phẩm phổ biến. Nhánh Lọc nội dung chiếm 40% trọng số đóng vai trò là cơ chế đệm đặc trưng thuộc tính, giúp bổ sung điểm số cho các sản phẩm mới hoặc người dùng có ít tương tác. Cơ chế này giải quyết rất hiệu quả bài toán khởi động lạnh (Cold-start) và nâng cao độ đa dạng cho danh sách gợi ý Top-K.

4.5. Luồng thực thi Mạng nơ-ron đồ thị LightGCN

Phương pháp Mạng nơ-ron đồ thị LightGCN đại diện cho hướng tiếp cận hiện đại nhất trong hệ tư vấn dựa trên cấu trúc đồ thị. Mô hình được cài đặt trong tệp **model_lightgcn.py** với kiến trúc phân cấp gồm hai Class kế thừa từ `nn.Module` của PyTorch: class **LightGCNConv** (lớp lan truyền Message Passing mức thấp) và class **LightGCNRecommender** (mô-đun bọc quản lý bảng nhúng toàn cục và gom tụ đa tầng).



Hình 7: Sơ Đồ Mối Quan Hệ Class LightGCNConv & LightGCNRecommender

Giải thích chi tiết sơ đồ:

Sơ đồ mô tả cấu trúc cài đặt mô hình LightGCN bằng PyTorch, gồm hai lớp chính là LightGCNConv và LightGCNRecommender. Cả hai lớp đều kế thừa từ nn.Module, là lớp cơ sở dùng để xây dựng và quản lý mô hình học sâu trong PyTorch.

Lớp LightGCNConv

LightGCNConv là lớp lan truyền thông tin mức thấp trên đồ thị User–Item. Lớp này không chứa tham số học được, không sử dụng ma trận trọng số, Bias hoặc hàm kích hoạt như GCN truyền thống. Nhiệm vụ chính của lớp là tổng hợp Embedding từ các đỉnh láng giềng.

Phương thức forward(x, edge_index) nhận vào ma trận Embedding x của toàn bộ User và Item cùng với edge_index biểu diễn các cạnh tương tác. Đầu tiên, phương thức tính bậc của từng đỉnh, tức là số lượng kết nối của mỗi User hoặc Item. Từ đó, hệ số chuẩn hóa được xác định theo công thức:

$$\frac{1}{\sqrt{|\mathcal{N}_u| + |\mathcal{N}_i|}}$$

Hệ số này giúp hạn chế ảnh hưởng của những User hoặc Item có quá nhiều tương tác. Sau đó, Embedding của nút nguồn được nhân với hệ số chuẩn hóa và cộng vào nút đích bằng phép scatter_add. Kết quả thu được là Embedding mới của tất cả các đỉnh sau một lớp lan truyền.

Lớp LightGCNRecommender

LightGCNRecommender là lớp quản lý mức cao, chịu trách nhiệm điều khiển toàn bộ mô hình LightGCN. Các thuộc tính chính gồm số lượng User, số lượng Item, kích thước Embedding và số lớp lan truyền K. Trong sơ đồ, mỗi User và Item được biểu diễn bằng vector có 32 chiều, còn K = 2, nghĩa là thông tin được lan truyền qua hai lớp.

Toàn bộ User và Item được lưu chung trong một bảng:

$$\text{Embedding} \in \mathbb{R}^{(M+N) \times 32}$$

Trong đó, M là số User và N là số Item. Lớp này đồng thời chứa một đối tượng LightGCNConv thông qua thuộc tính self.conv, thể hiện quan hệ thành phần giữa hai lớp.

Phương thức propagate(edge_index) lấy Embedding ban đầu và gọi LightGCNConv.forward() liên tiếp K lần. Với K = 2, mô hình thu được Embedding tại lớp 0, lớp 1 và lớp 2. Các vector này được xếp chồng và tính trung bình để tạo Embedding cuối cùng:

$$E^{(\text{final})} = \frac{1}{K+1} \sum_{k=0}^K E^{(k)}$$

Cách kết hợp này giúp mô hình giữ lại thông tin ban đầu, thông tin tương tác trực tiếp và các quan hệ gián tiếp trên đồ thị. Sau đó, Embedding chung được tách thành phần User và phần Item.

Phương thức `forward()` sử dụng Embedding cuối cùng để tính điểm phù hợp giữa User và Item bằng tích vô hướng. Điểm càng lớn thì Item càng được dự đoán phù hợp với sở thích của User.

Phương thức `compute_bpr_loss()` được sử dụng trong quá trình huấn luyện. Với mỗi User, mô hình nhận một Item dương đã tương tác và một Item âm chưa tương tác. Mục tiêu là làm cho điểm của Item dương lớn hơn điểm của Item âm. Hàm mất mát BPR được kết hợp với chuẩn hóa L_2 để giảm nguy cơ quá khớp.

Phương thức `predict_score_matrix(edge_index)` tính toàn bộ ma trận điểm:

$$S = E_U E_I^T$$

Trong đó, mỗi phần tử $S_{u,i}$ là điểm dự đoán giữa User u và Item i . Hệ thống loại bỏ các Item đã tương tác, sắp xếp điểm giảm dần và chọn các Item có điểm cao nhất để tạo danh sách gợi ý Top- K .

Trong quá trình huấn luyện, chuỗi lời gọi được thực hiện theo thứ tự: `train_eval.py` gọi `compute_bpr_loss()`, phương thức này gọi `propagate()`, sau đó `LightGCNConv.forward()` được thực hiện K lần. Cuối cùng, mô hình tính BPR Loss, thực hiện lan truyền ngược và dùng `optimizer.step()` để cập nhật các vector Embedding.

Luồng xử lý được hiện thực hóa qua 3 bước với các điểm ánh xạ lý thuyết và kỹ thuật cài đặt cụ thể như sau:

Bước 1: Xây dựng Đồ thị Lưỡng phân Vô hướng

- Định nghĩa Đồ thị Lưỡng phân $G = (U \cup I, E)$ trong đó tập đỉnh User (U) và tập đỉnh Item (I) giao nhau bằng rỗng ($U \cap I = \emptyset$), các cạnh $e = (u, i) \in E$ biểu diễn tương tác tích cực giữa người dùng u và sản phẩm i ở Mục 3.4 ở Chương III.
- Cài đặt trong mã nguồn:

- Mã hóa chỉ số nút toàn cục: Trong tệp **data_loader.py**, hệ thống thực hiện ánh xạ chỉ số đỉnh trên toàn đồ thị hợp $U \cup I$. Các M người dùng nhận chỉ số $0 \dots M - 1$, còn N sản phẩm nhận chỉ số nối tiếp từ $M \dots M + N - 1$ (**item_node_id = num_users + i**).
- Tạo danh sách cạnh 2 chiều: Mỗi tương tác $(u, i) \in \text{Train}$ tự động sinh ra hai cạnh ngược chiều để đảm bảo tính chất vô hướng của đồ thị: cạnh thuận $u \rightarrow (M + i)$ và cạnh ngược $(M + i) \rightarrow u$. Cấu trúc này được đóng gói thành Tensor danh sách cạnh **edge_index** kích thước $[2, 2|E|]$.
- Việc biểu diễn đồ thị dạng ma trận cạnh **edge_index** cho phép thuật toán tính toán lan truyền thông điệp hai chiều ($\text{User} \rightarrow \text{Item}$ và $\text{Item} \rightarrow \text{User}$) song song trên GPU mà không cần khởi tạo ma trận kề thừa không lồ kích thước $(M + N) \times (M + N)$, giúp tối ưu dung lượng bộ nhớ VRAM.

Bước 2: Lớp lan truyền thông điệp chuẩn hóa độ bậc (LightGCNConv)

- Lớp **LightGCNConv** đảm nhiệm việc lan truyền thông điệp 1-hop được chuẩn hóa đối xứng theo độ bậc. Với mỗi đỉnh u và i , công thức lan truyền được định nghĩa:

$$e_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u| \cdot |\mathcal{N}_i|}} e_i^{(k)}$$

$$e_i^{(k+1)} = \sum_{u \in \mathcal{N}_i} \frac{1}{\sqrt{|\mathcal{N}_i| \cdot |\mathcal{N}_u|}} e_u^{(k)}$$

- Trong phương thức **forward()**, hệ thống trích xuất đỉnh nguồn và đỉnh đích từ **edge_index** để tính độ bậc của từng đỉnh. Trọng số chuẩn hóa đối xứng cho mỗi cạnh được xác định bằng công thức

$$\frac{1}{\sqrt{\deg_{src} \times \deg_{dst}}}.$$

Cuối cùng, vector nhúng của đỉnh láng giềng được nhân với trọng số này và cộng dồn về đỉnh đích.

- Điểm đột phá của lớp **LightGCNConv** so với các kiến trúc GNN truyền thống là việc loại bỏ hoàn toàn ma trận trọng số W và hàm kích hoạt phi tuyến (như ReLU). Sự tối giản này giúp hạn chế hiện tượng quá tối ưu (over-parameterization) trên dữ liệu thưa, tập trung phản ánh trực tiếp sự tương đồng giữa các nút láng giềng.

Bước 3: Quản lý Bảng nhúng & Gom tụ đa tầng (LightGCNRecommender)

- Lớp **LightGCNRecommender** điều phối toàn bộ vòng lặp lan truyền và tổng hợp kết quả. Ban đầu, hệ thống khởi tạo một bảng nhúng chung **self.embedding** kích thước $(M + N) \times d$ (với $d = 32$) cho toàn bộ đồ thị, các giá trị vector $e^{(0)}$ được lấy mẫu ngẫu nhiên từ phân phối chuẩn $\mathcal{N}(0, 0.1)$.
- Quá trình lan truyền đa tầng được thực hiện qua phương thức **propagate()**. Hàm duy trì một vòng lặp $K = 2$ tầng, liên tục gọi lớp Conv để tính vector biểu diễn $e^{(k)}$ tại mỗi bước. Các vector từ các tầng $\{e^{(0)}, e^{(1)}, e^{(2)}\}$ được lưu trữ, xếp chồng thành Tensor 3 chiều và lấy trung bình cộng để tạo ra biểu diễn cuối cùng $e^{(final)}$:

$$e^{(final)} = \frac{1}{K + 1} \sum_{k=0}^K e^{(k)}$$

$$\hat{y}_{ui} = e_u^{(final)T} e_i^{(final)}$$

- Phép cộng trung bình này tích hợp cả tín hiệu trực tiếp (1-hop) và gián tiếp (2-hop), đồng thời là giải pháp triệt để ngăn chặn hiện tượng làm mờ thông tin (over-smoothing) khi số lượng tầng tăng lên.
- Cuối cùng, $e^{(final)}$ được phân tách lại thành ma trận nhúng người dùng $E_{User}^{(final)} \in \mathbb{R}^{M \times d}$ và ma trận sản phẩm $E_{Item}^{(final)} \in \mathbb{R}^{N \times d}$. Phương thức **predict_score_matrix()** thực hiện phép nhân ma trận toàn cục để tính ra ma trận điểm số dự đoán $\mathbf{S}_{LGCN} \in \mathbb{R}^{M \times N}$:

$$\mathbf{S}_{LGCN} = E_{User}^{(final)} \times E_{Item}^{(final)T}$$

Trả về toàn bộ ma trận điểm dự đoán $\mathbf{S}_{LGCN} \in \mathbb{R}^{M \times N}$ cho M người dùng và N sản phẩm.

- Phép cộng trung bình đa tầng $e^{(final)}$ đóng vai trò tích hợp cả thông tin ngữ cảnh trực tiếp (1-hop) lẫn ngữ cảnh gián tiếp (2-hop), đồng thời khắc phục triệt để hiện tượng Over-smoothing khi mạng đồ thị đi quá sâu.

4.6. Quy trình Huấn luyện và Đánh giá tổng thể

Kịch bản điều phối trung tâm của toàn bộ hệ thống được hiện thực hóa tại tệp `train_eval.py`, kết hợp chặt chẽ cùng thư viện đo lường `metrics.py`. Khối này đóng vai trò khép kín vòng lặp từ khâu tối ưu hóa Gradient (Train) cho đến khâu kiểm thử tính chính xác của danh sách Top-K (Test).

Luồng thực thi bao gồm 3 cơ chế cốt lõi:

4.6.1. Thuật toán Lấy mẫu âm (Negative sampling)

- Các mô hình học tham số (Phân rã ma trận và LightGCN) sử dụng hàm mất mát BPR Loss. Hàm này yêu cầu dữ liệu đầu vào dưới dạng bộ ba (u, i, j) , trong đó i là *Item* dương (đã tương tác) và j là *Item* âm (chưa tương tác)

- Hàm **sample_negative_items()** trong **train_eval.py** được gọi tại mỗi Epoch. Vòng lặp duyệt qua toàn bộ danh sách tương tác dương $(u, i) \in \text{Train}$. Với mỗi cặp, thuật toán rút ngẫu nhiên một sản phẩm j từ không gian N sản phẩm. Một vòng lặp while nội bộ sẽ kiểm tra bắt buộc điều kiện $j \notin I_{u+}^{\text{train}}$ để đảm bảo j thực sự là một mẫu âm đối với *User* u . Kỹ thuật này liên tục làm mới mẫu âm qua từng Epoch, giúp mô hình học được ranh giới phân loại xếp hạng một cách mạnh mẽ nhất.

4.6.2. Cơ chế Che lịch sử tương tác (Train masking)

- Để mô phỏng kịch bản thực tế (chỉ gợi ý các sản phẩm hoàn toàn mới cho người dùng), hệ thống phải loại bỏ các *Item* đã xuất hiện trong lịch sử huấn luyện ra khỏi danh sách đánh giá tập Test.
- Trước khi tiến hành sắp xếp điểm số dự đoán lấy Top-K, phương thức **evaluate_model()** trong tệp **metrics.py** tiến hành can thiệp trực tiếp vào ma trận điểm **S**. Cụ thể, thuật toán duyệt qua danh sách các *Item* nằm trong tập Train của *User* u và gán đè điểm số của chúng bằng Âm vô cực:

$$S_{u,i} = \begin{cases} -\infty, & \text{nếu } i \in I_{u+}^{\text{train}} \\ S_{u,i}, & \text{ngược lại} \end{cases}$$

- Việc gán điểm số dự đoán bằng $-\infty$ đảm bảo khi sử dụng các thuật toán sắp xếp mảng giảm dần (sort / topk), các sản phẩm cũ này sẽ tự động bị đẩy xuống vị trí chót cùng của bảng xếp hạng, tuyệt đối không thể lọt vào danh sách Top-K mang đi tính toán độ đo.

4.6.3. Chấm điểm Độ đo xếp hạng Top-K

- Sau khi áp dụng cơ chế Masking, hệ thống tiếp tục trích xuất K phần tử có điểm số cao nhất để tạo thành danh sách dự đoán **predicted_rank_list**. Danh sách này được đối chiếu trực tiếp với tập đích thực tế T_u (Test Set) của người dùng đó.
- Các hàm tính toán chuyên biệt (như **precision_at_k()**, **ndcg_at_k()**...) sẽ thực thi các phép toán trên mảng để trả về điểm số cục bộ cho từng người dùng dựa trên hệ thống công thức tiêu chuẩn (Precision@K, Recall@K, NDCG@K, MRR@K, Coverage). Cuối cùng, hệ thống tính trung bình cộng (mean) tổng điểm của tất cả người dùng có trong tập Test và tự động xuất ra bảng báo cáo, cung cấp cái nhìn so sánh trực quan về hiệu năng giữa 4 mô hình.

CHƯƠNG V: THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Thiết lập thực nghiệm và Đặc tả các Bộ dữ liệu

5.1.1 Tổng quan 3 bộ dữ liệu chuẩn quốc tế:

Thực hiện yêu cầu thực nghiệm đa dạng miền ứng dụng của giảng viên hướng dẫn, đề án lựa chọn 3 bộ dữ liệu công khai uy tín được công bố rộng rãi trong giới nghiên cứu quốc tế:

- MovieLens Latest Small: Bộ dữ liệu do Tập đoàn nghiên cứu GroupLens công bố, đại diện cho miền nội dung điện ảnh.
- Last.fm hetrec2011: Bộ dữ liệu nghe nhạc từ nền tảng Last.fm công bố tại Hội nghị HetRec 2011. Dữ liệu đại diện cho miền giải trí âm nhạc với mật độ nghe nghệ sĩ và các thẻ gắn User Tags.
- Amazon Musical Instruments: Bộ dữ liệu mua sắm thuộc chuỗi Amazon Review Datasets công bố bởi Stanford SNAP Lab. Dữ liệu đại diện cho miền Thương mại điện tử với các nhận xét văn bản thô.

5.1.2 Bảng đặc tả thông số thống kê đối chiếu:

Toàn bộ 3 bộ dữ liệu được chuyển về dạng Tương tác ẩn (Implicit Feedback) bằng cách lọc các tương tác tích cực (rating ≥ 3.0 sao hoặc số lần nghe ≥ 1). Các thông số thống kê đối chiếu giữa 3 bộ dữ liệu được tổng hợp như sau:

Thuộc tính thống kê	MovieLens	Last.fm	Amazon Instruments
Nguồn dữ liệu	GroupLens (Univ. Minnesota)	HetRec 2011 (ACM RecSys)	Stanford SNAP Lab
Số User	609 Users	1,892 Users	1,429 Users
Số Item	8,452 Phim	17,632 Nghệ sĩ	900 Sản phẩm
Tương tác Train (80%)	65,175 lượt	74,267 lượt	7,835 lượt
Tương tác Val (10%)	7,909 lượt	9,283 lượt	979 lượt
Tương tác Test (10%)	8,679 lượt	9,284 lượt	980 lượt
Tổng Tương tác tích cực ($ E $)	81,763 lượt	92,834 lượt	9,794 lượt
Mật độ tương tác (Density)	1.3781%	0.2783%	0.7978%
Độ thưa ma trận (Sparsity)	98.6219%	99.7217%	99.2022%
Đặc trưng thuộc tính	19 Thẻ loại	Thẻ gắn (User Tags)	Nhận xét (Review Text)

5.1.3 Quy trình tiền xử lý dữ liệu:

- Bước 1 - Loại tương tác âm: Chuyển đổi dữ liệu rating tương minh thành tương tác nhị phân (1 đối với rating ≥ 3.0 hoặc weight ≥ 1 , và 0 đối với rating < 3.0).
- Bước 2 - Mã hóa định danh hai chiều (Node ID Mapping): Ánh xạ User ID thô sang chỉ số mảng liên tục u in $[0, M-1]$ và Item ID sang i in $[0, N-1]$. Đối với mạng LightGCN, chỉ số đỉnh sản phẩm được tịnh tiến thành $\text{item_node_id} = M + i$ để tạo đồ thị lưỡng phân hợp nhất $G = (U \cup I, E)$.
- Bước 3 - Trích xuất đặc trưng TF-IDF: Duyệt qua tập thuộc tính văn bản (thẻ loại, thẻ tag, nhận xét), tính trọng số TF-IDF và chuẩn hóa độ dài Euclidean (L2-norm) trên từng hàng để thu ma trận đặc trưng $\mathbf{V}_{\text{Item}} \in \mathbb{R}^{N \times d_{\text{feat}}}$.
- Bước 4 - Phân tách tập dữ liệu: Xáo trộn ngẫu nhiên (random_seed = 42) gom theo từng User chia theo tỷ lệ 80% Train / 10% Validation / 10% Test, đảm bảo mỗi User có ít nhất 1 tương tác trong tập Train.

5.1.4 Thiết lập Tham số hệ thống:

Toàn bộ 4 mô hình được cấu hình thống nhất trên cả 3 bộ dữ liệu theo các tham số quy chuẩn như sau:

Tham số	Giá trị thiết lập	Vai trò kỹ thuật
Kích thước chiều ẩn (d)	32	Kích thước $d=32$ giúp vector nhúng học đủ đặc trưng sở thích mà không bị overfitting trên dữ liệu thưa.
Số tầng lan truyền (K)	2 tầng	Lan truyền 2 bước giúp thu thập láng giềng gián tiếp mà không bị Over-smoothing.
Tốc độ học (lr)	0.01	Tối ưu hóa bằng thuật toán Adam cho tốc độ hội tụ BPR Loss nhanh và ổn định.
Kích thước lô (Batch Size)	2048	Xử lý song song các bộ ba (u, i, j)
Hệ số phạt chuẩn hóa (reg)	1e-4	Trọng số phạt L2 Regularization tránh bùng nổ trọng số khi cập nhật Gradient.
Trọng số mô hình Lai (alpha)	0.6	Dành 60% ưu tiên cho tín hiệu hành vi MF và 40% cho đặc trưng nội dung CB.
Số Epoch huấn luyện	50 Epochs	Hàm mất mát BPR Loss giảm mạnh và hội tụ ổn định trên cả 3 bộ dữ liệu.

5.2. Hệ thống các Độ đo Đánh giá Tiêu chuẩn

Chất lượng xếp hạng danh sách Top-K gợi ý cho từng người dùng u trong tập Test được đo lường qua 6 chỉ số định lượng tiêu chuẩn:

- **Precision@K:** Độ chính xác Top-K, đo lường tỷ lệ phần trăm các sản phẩm dự đoán đúng nằm trong danh sách K sản phẩm được gợi ý:

$$\text{Precision@K} = \frac{|\text{Top-K}(u) \cap I_{\text{test}}(u)|}{K}$$

- **Recall@K:** Độ bao phủ sở thích Top-K, đo lường tỷ lệ phần trăm các sản phẩm thực tế người dùng thích trong tập Test được hệ thống tìm ra thành công:

$$\text{Recall@K} = \frac{|\text{Top-K}(u) \cap I_{\text{test}}(u)|}{|I_{\text{test}}(u)|}$$

- **NDCG@K:** Đánh giá chất lượng thứ hạng xếp hạng. Sản phẩm đúng xuất hiện ở vị trí càng cao (gần đầu danh sách) càng đóng góp điểm số lớn hơn:

$$\text{DCG@K} = \sum_{r=1}^K \frac{\mathbb{I}(r \in I_{\text{test}}(u))}{\log_2(r+1)}, \quad \text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

(trong đó $\mathbb{I}(\cdot)$ là hàm chỉ thị, trả về 1 nếu sản phẩm tại vị trí r thuộc tập Test, ngược lại bằng 0).

- **MRR@K:** Đánh giá độ nhạy thứ hạng thông qua nghịch đảo vị trí xuất hiện của sản phẩm đúng đầu tiên trong danh sách gợi ý:

$$\text{MRR@K} = \frac{1}{\text{rank}_{\text{first}}}$$

- **HitRate@K:** Tỷ lệ gợi ý trúng, trả về 1.0 nếu người dùng nhận được ít nhất 1 sản phẩm đúng trong danh sách Top-K, ngược lại bằng 0.0:

$$\text{HitRate@K} = \begin{cases} 1.0, & \text{nếu } |\text{Top-K}(u) \cap I_{\text{test}}(u)| \geq 1 \\ 0.0, & \text{ngược lại} \end{cases}$$

- **Coverage@K:** Đo lường tỷ lệ phần trăm các sản phẩm độc bản trong toàn bộ kho N sản phẩm được hệ thống gợi ý cho tất cả người dùng:

$$\text{Coverage@K} = \frac{|\bigcup_{u \in U} \text{Top-K}(u)|}{N}$$

5.3. Kết quả thực nghiệm trên MovieLens

Đặc tả miền Phim ảnh: Bộ dữ liệu MovieLens Lastest Small chứa 609 Users, 8,452 Items (Phim), 81,763 Tương tác tích cực với độ thưa ma trận 98.6219%. Đặc trưng thuộc tính gồm 19 thể loại phim được trích xuất ma trận TF-IDF.

Kết quả thực nghiệm ngưỡng Top-10

Phương pháp Tư vấn	Precision@10	Recall@10	NDCG@10	MRR@10	HitRate@10	Coverage@10
Content-Based (CB)	0.0039	0.0060	0.0056	0.0120	0.0361	0.1542
Matrix Factorization (MF)	0.0883	0.0969	0.1173	0.2416	0.5402	0.1158
Hybrid Late Fusion (alpha=0.6)	0.0747	0.0869	0.1005	0.2020	0.4844	0.1155
LightGCN (GNN - Mạng Đồ thị)	0.1143	0.1244	0.1524	0.2878	0.6158	0.0712

Kết quả thực nghiệm ngưỡng Top-20

Phương pháp Tư vấn	Precision@20	Recall@20	NDCG@20	MRR@20	HitRate@20	Coverage@20
Content-Based (CB)	0.0040	0.0092	0.0070	0.0144	0.0706	0.2433
Matrix Factorization (MF)	0.0742	0.1618	0.1356	0.2516	0.6814	0.1577
Hybrid Late Fusion (alpha=0.6)	0.0608	0.1357	0.1137	0.2117	0.6207	0.1685
LightGCN (GNN - Mạng Đồ thị)	0.0965	0.2011	0.1742	0.2969	0.7438	0.1080

Nhận xét:

Mạng nơ-ron đồ thị LightGCN đạt kết quả cao nhất toàn diện ở mọi chỉ số chính xác (Recall@20 = 20.11%, NDCG@20 = 0.1742, HitRate@20 = 74.38%).

Lý do bản chất: Trong khi MF chỉ tính tích vô hướng 1-hop trực tiếp, LightGCN biểu diễn bài toán dưới dạng Đồ thị Lượng phân User-Item và thực hiện lan truyền Message Passing qua 2 tầng láng

giềng. Nhờ đó, vector nhúng thu được $\mathbf{e}^{(final)}$ tích hợp được ngữ cảnh láng giềng gián tiếp (2-hops), học được quan hệ cấu trúc ẩn cực kỳ hiệu quả trên ma trận thưa 98.62%.

5.4. Kết quả thực nghiệm trên Last.fm

Đặc tả miền Âm nhạc: Bộ dữ liệu Last.fm chứa 1,892 Users, 17,632 Items, 92,834 Tương tác nghe nhạc với độ thưa ma trận 99.7217%. Đặc trưng thuộc tính gồm các User Tags do cộng đồng định nghĩa.

Kết quả thực nghiệm ngưỡng Top-10

Phương pháp Tư vấn	Precision@10	Recall@10	NDCG@10	MRR@10	HitRate@10	Coverage@10
Content-Based (CB)	0.0332	0.0680	0.0593	0.1111	0.2654	0.1751
Matrix Factorization (MF)	0.0570	0.1159	0.1007	0.1821	0.4098	0.1273
Hybrid Late Fusion (alpha=0.6)	0.0737	0.1494	0.1310	0.2342	0.5186	0.1527
LightGCN (GNN - Mạng Đồ thị)	0.0843	0.1700	0.1566	0.2816	0.5653	0.0953

Kết quả thực nghiệm ngưỡng Top-20

Phương pháp Tư vấn	Precision@20	Recall@20	NDCG@20	MRR@20	HitRate@20	Coverage@20
Content-Based (CB)	0.0281	0.1144	0.0791	0.1204	0.3981	0.2575
Matrix Factorization (MF)	0.0426	0.1729	0.1249	0.1909	0.5377	0.1845
Hybrid Late Fusion (alpha=0.6)	0.0562	0.2279	0.1641	0.2449	0.6720	0.2232
LightGCN (GNN - Mạng Đồ thị)	0.0603	0.2436	0.1878	0.2908	0.6959	0.1551

Nhận xét:

Trực giác âm nhạc có tính lặp lại và cộng đồng cao. Trên tập Last.fm, LightGCN bứt phá mạnh mẽ đạt $\text{Recall@20} = 24.36\%$ và $\text{HitRate@20} = 69.59\%$. Nhờ cấu trúc đồ thị hình thành các cụm tương đồng liên kết mạnh, cơ chế thu gom thông điệp láng giềng đối xứng giúp LightGCN ghi nhận gu âm nhạc của cộng đồng một cách tối ưu nhất.

5.5. Kết quả thực nghiệm trên Amazon Musical Instruments

Đặc tả miền Thương mại điện tử: Bộ dữ liệu Amazon Musical Instruments chứa 1,429 Users, 900 Items (Thiết bị âm nhạc), 9,794 Tương tác tích cực với độ thưa ma trận 99.2022%. Đặc trưng thuộc tính là các đoạn văn bản nhận xét sản phẩm (Review Text).

Kết quả thực nghiệm ngưỡng Top-10

Phương pháp Tư vấn	Precision@10	Recall@10	NDCG@10	MRR@10	HitRate@10	Coverage@10
Content-Based (CB)	0.0072	0.0470	0.0227	0.0189	0.0707	0.2900
Matrix Factorization (MF)	0.0072	0.0478	0.0290	0.0283	0.0714	0.9367
Hybrid Late Fusion ($\alpha=0.6$)	0.0091	0.0594	0.0359	0.0357	0.0896	0.5278
LightGCN (GNN - Mạng Đồ thị)	0.0175	0.1125	0.0681	0.0668	0.1672	0.8944

Kết quả thực nghiệm ngưỡng Top-20

Phương pháp Tư vấn	Precision@20	Recall@20	NDCG@20	MRR@20	HitRate@20	Coverage@20
Content-Based (CB)	0.0070	0.0889	0.0345	0.0230	0.1302	0.3367
Matrix Factorization (MF)	0.0059	0.0773	0.0374	0.0313	0.1141	0.9889

Hybrid Late Fusion (alpha=0.6)	0.0083	0.1077	0.0496	0.0404	0.1575	0.6289
LightGCN (GNN - Mạng Đồ thị)	0.0125	0.1603	0.0818	0.0710	0.2295	0.9800

Nhận xét:

Trên bộ dữ liệu E-commerce có số lượng sản phẩm ít hơn nhưng độ thưa cao, mô hình Lai thể hiện rõ vai trò giải quyết bài toán Cold-Start. Khi kết hợp 40% trọng số từ Lọc nội dung (alpha = 0.6), chỉ số Coverage@20 của Hybrid đạt 62.89% (cao hơn nhiều so với CB 33.67% và giữ cân bằng tối ưu so với MF, LightGCN), giúp nâng cao khả năng gợi ý các sản phẩm mới có ít tương tác.

5.6. Phân tích So sánh Tổng hợp và Đánh giá Tính Tổng quát

- **Phân tích đặc tính biểu diễn của các mô hình cơ sở:**
 - Các mô hình truyền thống như Matrix Factorization chỉ khai thác tương tác 1-hop trực tiếp, dẫn đến giới hạn về khả năng biểu diễn ngữ cảnh láng giềng gián tiếp.
 - Mô hình Content-Based tuy đóng góp độ bao phủ (Coverage@20 đạt từ 24.33% đến 33.67%) nhờ thuộc tính văn bản, nhưng lại có độ chính xác tương tác thấp.
 - Mô hình Hybrid Late Fusion thể hiện khả năng dung hòa tương đối giữa hành vi cộng đồng và thuộc tính sản phẩm, đóng vai trò là bước đệm so sánh hiệu năng.
- **Khả năng tổng quát hóa mạnh mẽ của mô hình đề xuất LightGCN:** Khi thực nghiệm trên cả 3 miền ứng dụng khác nhau (Phim ảnh, Âm nhạc, Thương mại điện tử), giải pháp Mạng nơ-ron đồ thị LightGCN luôn đạt kết quả cao nhất và khẳng định sự vượt trội tuyệt đối về các chỉ số độ chính xác xếp hạng (đặc biệt là Recall@20 và NDCG@20). Điều này chứng minh tính đúng đắn và ưu thế vượt trội của cơ chế lan truyền thông điệp (Message Passing) trên Đồ thị Lường phân trong việc khai phá các cấu trúc ẩn bậc cao trên ma trận dữ liệu cực kỳ thưa thớt